SEARCH IN SNAKE

Laporan Project 1

IF5150 Advanced Artificial Intelligence



Disusun oleh

Kelompok 4:

Gina Rahayu Meilani	23526007
Arista Widya Andhika Akbar	23526012
Eka Miftakhul Rachmawati	23526049

Repositori GitHub: [https://github.com/ekamifta/Tugas-Besar-AI-Lanjut-Snake.git]

PROGRAM STUDI INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

BANDUNG

TAHUN AKADEMIK 2026/2027 SEMESTER 1

Daftar Isi

Daftar Isi **2**
Daftar Tabel **3**
Daftar Gambar **3**
Daftar Lampiran **3**
BAB 1
Pendahuluan **3**
1.1 Deskripsi Singkat Project 3
1.2 Formulasi Masalah (Search Problem Formulation) 4
1.2.1 Single Player Mode 4
1.2.2 Multiplayer Mode (Adversarial Search) 5
1.3 Pembagian Tugas dan Kontribusi Anggota Group 6
BAB 2
Implementasi Algoritma **7**
2.1 Single Player Mode 7
2.1.1 Algoritma 1: Depth-First Search (DFS) 7
2.1.2 Algoritma 2: Breadth-First Search (BFS) 9
2.1.3 Algoritma 3: Uniform-Cost Search (UCS)
2.1.4 Algoritma 4: A* Search 14
2.2 Multiplayer Mode 17
2.2.1 Minimax 17
2.2.2 Alpha-Beta Pruning 22
2.3 Algoritma Bonus - Algoritma: Bidirectional Search 26
BAB 3
Eksperimen dan Evaluasi **30**
3.1 Evaluasi Single Player Mode 30
3.1.1 Hasil Pengujian Single Player 30
3.1.2 Analisis dan Pembahasan Single Player 30
3.2 Evaluasi Multiplayer Mode 33
3.2.1 Hasil Pengujian Multiplayer 33
3.2.2 Analisis dan Pembahasan Multiplayer 34
Daftar Referensi **39**
Lampiran **41**

Daftar Tabel

Tabel 1. Pembagian Tugas 7
Tabel 2. Definisi Metrics DFS 10
Tabel 3. Definisi Metrics BFS 12
Tabel 4. Definisi Metrics UCS 14
Tabel 5. Definisi Metrics A* 17
Tabel 6. Parameter Minmax 19
Tabel 7. Tabel Kondisi 21
Tabel 8. Definisi Metrics Minmax 22
Tabel 9. Definisi Metrics Alpha-Beta 25
Tabel 10. Definisi Metrics Bidirectional Search 29
Tabel 11. Tabel Pengujian Single Player 31
Tabel 12. Tabel Pengujian Multi Player 34
Tabel 13. Tabel Pengujian Multi Player 36

Daftar Gambar

Gambar 1. DFS (GeeksforGeeks, Update 2026) 9
Gambar 2. BFS (GeeksforGeeks, Update 2026) 12
Gambar 3. UCS 14
Gambar 4. A* (GeeksforGeeks, Update 2026) 18
Gambar 5. Minimax (GeeksforGeeks, Update 2026) 21
Gambar 6. Alpha-Beta (GeeksforGeeks, Update 2026) 24
Gambar 7. Alpha-Beta Pruning(GeeksforGeeks, Update 2026) 27
Gambar 8. Bidirectional Search (GeeksforGeeks, Update 2026) 28

Daftar Lampiran

Lampiran Depth-First Search (DFS)	44
Lampiran Breadth-First Search (BFS)	46
Lampiran Bidirectional Search	48
Lampiran UCS	50
Lampiran A*	52
Lampiran MINIMAX (as Player 1)	54
Lampiran AlphaBeta Pruning	61

BAB 1
Pendahuluan

1.1 Deskripsi Singkat Project

Search in Snake merupakan project implementasi dan evaluasi algoritma *searching* pada permainan Snake untuk mata kuliah IF5150 Advanced Artificial Intelligence. Proyek ini bertujuan untuk membandingkan berbagai algoritma pencarian dalam menemukan jalur menuju apel. Permainan mencakup dua mode, yaitu *Single Player* dan *Multiplayer*. Pada *Single Player*, algoritma yang digunakan adalah *classical* atau *basic search*, yaitu DFS, BFS, UCS, A*, serta Bidirectional Search. Sementara mode *Multiplayer* menerapkan *adversarial search*, yaitu Minimax dan Alpha-Beta Pruning untuk memilih langkah berdasarkan kemungkinan pergerakan pemain dan respons lawan. Evaluasi dilakukan menggunakan metrik seperti jumlah makanan terkumpul, *success rate*, *states expanded*, *states generated*, *path length*, *path cost*, dan *execution time*. Hasil pengujian digunakan untuk menganalisis kelebihan dan keterbatasan setiap algoritma berdasarkan kelengkapan pencarian, optimalitas, dan efisiensi komputasi.

1.2 Formulasi Masalah (*Search Problem Formulation*)

Permainan **Search in Snake** pada proyek ini berjalan di grid (contoh: 20×20) dengan koordinat (x, y), dengan titik (0, 0) di kiri atas. Ular bergerak satu petak per giliran ke salah satu dari empat arah, dan tidak bisa berhenti hingga ***Move Limit*** tercapai. Formulasi dibagi menjadi dua mode sesuai jenis algoritmanya. Detail tiap algoritma dibahas di subbab masing-masing.

1.2.1 *Single Player Mode*

Proyek ini mensyaratkan pemilihan 4 (empat) algoritma pada mode *Single Player Mode*. Algoritma yang dipilih oleh kelompok kami adalah algoritma pencarian berikut: Depth-First Search (DFS), Breadth-First Search (BFS), Uniform-Cost Search (UCS), dan A*. Hanya ada satu ular yang bergerak sendiri di arena, dan masalahnya dirumuskan sebagai pencarian jalur dari kondisi awal ke kondisi tujuan.

- *Initial State*: Kondisi permainan saat giliran ular dimulai, yaitu posisi kepala ular, urutan posisi badan ular dari kepala sampai ekor, arah hadap saat ini, dan

konfigurasi seluruh makanan (apel) yang ada di grid. Posisi kepala menjadi akar (*root*) pohon pencarian.

- *Actions*: Himpunan arah gerak yang valid: Up, Down, Left, Right, dengan vektor (0,-1), (0,1), (-1,0), (1,0). Asumsi: Urutan aksi mengikuti urutan definisi enum Direction (Up, Down, Left Right).
- ***Transition Model***: Setelah suatu aksi diambil, kepala berpindah ke petak tujuan (x+dx, y+dy), dan badan mengikuti: ekor dilepas satu petak dan kepala baru ditambahkan di depan. Jika petak tujuan berisi makanan, panjang ular bertambah satu dan makanan tersebut hilang dari konfigurasi makanan.
- ***Goal Test***: Tercapai ketika **seluruh makanan telah terkumpul**, yaitu konfigurasi makanan kosong. Namun, pencarian akan dihentikan jika telah mencapai ***Move Limit***.
- ***Path Cost***: Asumsi: setiap langkah bernilai **1.**

1.2.2 *Multiplayer Mode (Adversarial Search)*

Project ini mensyaratkan pemilihan 2 (dua) algoritma pada mode *Multiplayer Mode*. Algoritma yang dipilih oleh kelompok kami adalah algoritma pencarian berikut: Minimax dan Alpha-Beta Pruning. Dua ular bermain di arena yang sama dan bergantian mengambil giliran, sehingga permainan dimodelkan sebagai *zero-sum game*: keuntungan satu pihak adalah kerugian pihak lain.

- ***Players & Terminal States***:
  - **Player MAX** adalah ular pemain yang dikendalikan algoritma (AI) dan memaksimalkan nilai utilitas.
  - **Player MIN** adalah ular lawan (*human* atau AI lain) yang diasumsikan memilih langkah terbaik untuk dirinya, yaitu meminimalkan nilai utilitas MAX. Giliran bergantian antara MAX dan MIN, dengan 3 layer.

  Referensi: Adversarial Search Algorithms in Artificial Intelligence (AI) - GeeksforGeeks

1.3 Pembagian Tugas dan Kontribusi Anggota Group

(Setiap anggota wajib mengimplementasikan minimal 2 algoritma)

Tabel 1. Pembagian Tugas

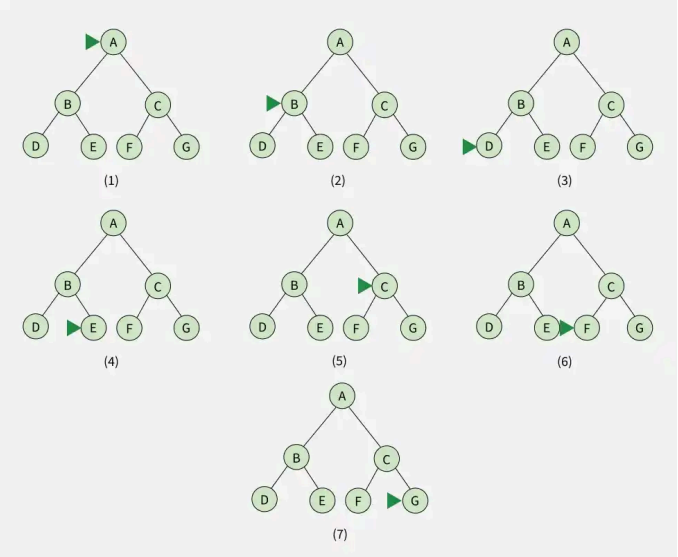
NIM	Nama Anggota	Algoritma yang Diimplementasikan	Kontribusi Laporan & Tugas Lain
23526007	Gina Rahayu Meilani	A* dan Alpha-Beta Pruning	Testing program, menambah metric A* dan Alpha-beta Pruning, merapikan laporan
23526012	Arista Widya Andhika Akbar	BFS, DFS, Bidirectional Search	Testing program, menambah metric BFS, DFS, dan Bidirectional Search, merapikan laporan
23526049	Eka Miftakhul Rachmawati	UCS, Minimax	Merging Code Request, Implementasi output Metrics, Pembuatan Template Laporan, Testing code

BAB 2

Implementasi Algoritma

2.1 *Single Player Mode*

2.1.1 Algoritma 1: Depth-First Search (DFS)



Gambar 1. DFS (GeeksforGeeks, Update 2026)

Formulasi Masalah

- ***State*:** Posisi kepala ular dalam x, y dengan posisi apel yang tidak berubah-ubah sejak pertama kali menjalankan game.
- ***Initial state***: Posisi kepala ular saat awal game yaitu snake.body[0], movement masih kosong, serta stack dan visited masih berisi kepala ular saat awal game saja.
- ***Goal state***: Posisi kepala ular sama dengan posisi apel.
- ***Actions***: Ular dapat bergerak ke arah UP, DOWN, LEFT, RIGHT.
- ***Transition***: Transisi terjadi ketika kepala ular beserta badannya bergerak satu kotak setelahnya sesuai arah yang diperbolehkan. Transisi posisi x, y adalah x+dx, y+dy.
- ***Legal move***: Arah yang diperbolehkan pada ular yaitu UP, DOWN, LEFT, RIGHT, posisi ular x tidak boleh lebih dari jumlah kolom, posisi ular y tidak boleh lebih dari jumlah baris, tidak boleh bergerak ke arah badan ular.
- ***Path cost***: Untuk DFS secara teori tidak ada path cost, tetapi pada algoritma yang dibuat diasumsikan adalah 1 untuk setiap langkah. g(n) = jumlah langkah dari initial state hingga final state.

- ***Obstacle*:** Dinding papan dan badan ular.

Deskripsi Algoritma

Depth-First Search (DFS) merupakan algoritma *searching* berdasarkan kedalaman node bila diasumsikan state space adalah graph. Tanpa adanya cost dan hanya berdasarkan jumlah langkah, DFS akan memilih state yang terdalam terlebih dahulu. DFS menggunakan stack untuk menyimpan *state* yang akan diperiksa yaitu berisi kepala ular x, y, dan movement yang sudah dicari. Stack berdasarkan Last-In First-Out sehingga *state* yang diproses lebih dulu adalah *state* yang berada di posisi stack terakhir. Supaya state yang sudah diproses ini tidak kembali masuk ke dalam stack dan menyebabkan *infinite loop*, ada visited untuk menyimpan posisi yang sudah dikunjungi (GeeksforGeeks, Update 2026).

Tahapan algoritma adalah sebagai berikut.

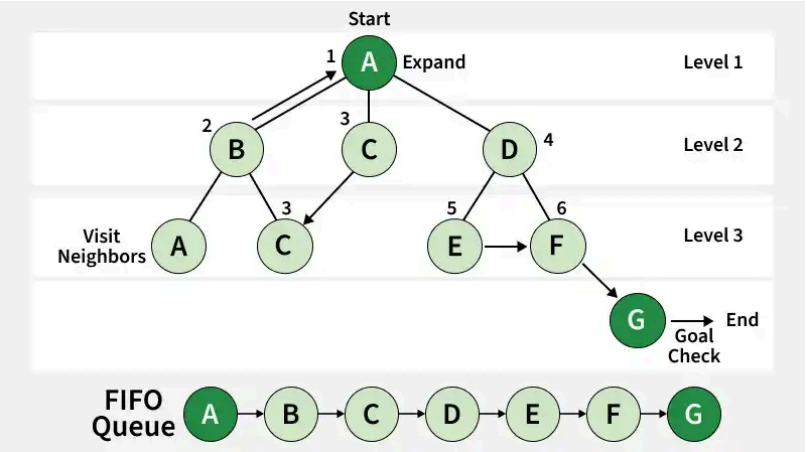
1. Inisialisasi *searching*: posisi kepala ular dan apel random saat awal game adalah initial state, inisialisasi queue dengan mengisi stack[0] = posisi kepala ular awal dan movement kosong, posisi kepala ular juga dimasukkan ke dalam visited.
2. Ambil *state* dari stack: menggunakan stack.pop() untuk mengambil dan menghapus state paling terakhir/atas pada stack.
3. Menghitung expanded state jika movement tidak kosong. *Initial state* tidak dihitung sebagai expanded karena belum ada movement.
4. Jika posisi kepala ular tidak sama dengan posisi apel yang berarti apel belum ditemukan, DFS akan memeriksa setiap arah. Langkah arah yang tidak termasuk dalam *legal move* akan dilewati. Memeriksa setiap arah ini akan menghasilkan *successor state*.
5. Memeriksa posisi baru dan batas kolom dan baris: bila posisi baru atau Transition masih di dalam papan, generated state akan bertambah. Jika posisi baru ini belum dikunjungi sebelumnya, sekarang dikunjungi, akan dimasukkan ke dalam visited, lalu state baru dimasukkan ke dalam stack.
6. *Searching* berhenti ketika posisi apel sama dengan posisi kepala ular atau apel ditemukan atau saat posisi stack kosong.
7. Menentukan gerakan atau solusi untuk ular dengan mengembalikan arah

pertama dari movement tersebut.

Tabel 2. Definisi Metrics DFS

Metrics	Definisi
Number of food items collected	Jumlah apel yang dimakan Snake.
Success rate	Rasio jumlah apel yang dimakan dengan jumlah apel yang ditetapkan dalam setup.
Generated	Jumlah *successor* posisi yang berada di dalam batas papan, termasuk posisi yang sudah pernah dikunjungi.
Expanded	Jumlah *state* selain *initial state* yang diproses sebelum proses *searching* selesai untuk menghasilkan *successor*.
Evaluated	Jumlah *state* yang diperiksa oleh algoritma. Evaluated sejumlah expanded karena *initial state* tidak dihitung dan state lainnya dihitung sebagai expanded.
Path Length	Jumlah gerakan pada jalur solusi yang ditemukan.
Path Cost	Seharusnya DFS tidak memiliki cost, tetapi untuk perhitungan metrics ini diasumsikan 1 cost untuk setiap langkah sehingga path_cost = path_length.
Execution time	Waktu eksekusi pencarian dalam milidetik, diukur menggunakan time.perf_counter()*, dihitung dengan selisih waktu sebelum dan sesudah pencarian.

2.1.2 Algoritma 2: Breadth-First Search (BFS)



Gambar 2. BFS (GeeksforGeeks, Update 2026)

Formulasi Masalah

- ***State***: Posisi kepala ular dalam x, y dengan posisi apel yang tidak berubah-ubah sejak pertama kali menjalankan game
- ***Initial state***: Posisi kepala ular saat awal game yaitu snake.body[0], movement masih kosong, serta queue dan visited masih berisi kepala ular saat awal game saja
- ***Goal state***: Posisi kepala ular sama dengan posisi apel
- ***Actions***: Ular dapat bergerak ke arah UP, DOWN, LEFT, RIGHT
- ***Transition***: Transisi terjadi ketika kepala ular beserta badannya bergerak satu kotak setelahnya sesuai arah yang diperbolehkan. Transisi posisi x, y adalah x+dx, y+dy.
- ***Legal move***: Arah yang diperbolehkan pada ular yaitu UP, DOWN, LEFT, RIGHT, posisi ular x tidak boleh lebih dari jumlah kolom, posisi ular y tidak boleh lebih dari jumlah baris, tidak boleh bergerak ke arah badan ular
- ***Path cost***: Untuk BFS secara teori tidak ada path cost, tetapi pada algoritma yang dibuat diasumsikan adalah 1 untuk setiap langkah. g(n) = jumlah langkah dari initial state hingga final state
- ***Obstacle*****:** Dinding papan dan badan ular

Deskripsi Algoritma

Breadth-First Search (BFS) merupakan algoritma *searching* berdasarkan kelebaran *node* bila diasumsikan *state space* adalah *graf*. Tanpa adanya *cost* dan hanya berdasarkan jumlah langkah, BFS akan memilih state yang terdekat dengan *initial state*.

BFS menggunakan queue (antrian) untuk menyimpan state yang akan diperiksa yaitu berisi kepala ular x, y, dan movement yang sudah dicari. Queue berdasarkan First-In First-Out sehingga state yang diproses lebih dulu adalah state yang berada di posisi queue[0]. Supaya *state* yang sudah diproses ini tidak kembali masuk ke dalam queue dan menyebabkan *infinite loop*, ada visited untuk menyimpan posisi yang sudah dikunjungi (GeeksforGeeks, Update 2026).

Tahapan algoritma adalah sebagai berikut.

1. Inisialisasi *searching*: posisi kepala ular dan apel random saat awal game adalah *initial state*, inisialisasi queue dengan mengisi queue[0] = posisi kepala ular awal dan movement kosong, posisi kepala ular juga dimasukkan ke dalam visited.
2. Ambil *state* dari *queue*: menggunakan queue.pop(0) untuk mengambil dan menghapus state paling depan pada queue.
3. Menghitung expanded state jika movement tidak kosong. *Initial state* tidak dihitung sebagai expanded karena belum ada movement.
4. Jika posisi kepala ular tidak sama dengan posisi apel yang berarti apel belum ditemukan, BFS akan memeriksa setiap arah. Langkah arah yang tidak termasuk dalam legal move akan dilewati. Memeriksa setiap arah ini akan menghasilkan *successor state*.
5. Memeriksa posisi baru dan batas kolom dan baris: bila posisi baru atau Transition masih di dalam papan, generated state akan bertambah. Jika posisi baru ini belum dikunjungi sebelumnya, sekarang dikunjungi, akan dimasukkan ke dalam visited, lalu state baru dimasukkan ke dalam queue.
6. *Searching* berhenti ketika posisi apel sama dengan posisi kepala ular atau apel ditemukan atau saat posisi queue kosong.
7. Menentukan gerakan atau solusi untuk ular dengan mengembalikan arah pertama dari movement tersebut.

Tabel 3. Definisi Metrics BFS

Metrics	**Definisi**
Number of food items collected	Jumlah apel yang dimakan Snake.

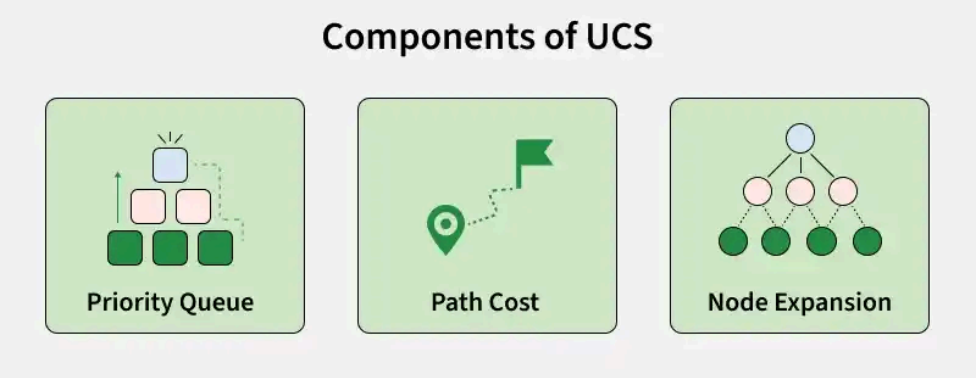
Metrics	Definisi
Success rate	Rasio jumlah apel yang dimakan dengan jumlah apel yang ditetapkan dalam setup.
Generated	Jumlah successor posisi yang berada di dalam batas papan, termasuk posisi yang sudah pernah dikunjungi.
Expanded	Jumlah state selain initial state yang diproses sebelum proses searching selesai untuk menghasilkan successor.
Evaluated	Jumlah state yang diperiksa oleh algoritma. Evaluated sejumlah expanded karena initial state tidak dihitung dan state lainnya dihitung sebagai expanded.
Path Length	Jumlah gerakan pada jalur solusi yang ditemukan.
Path Cost	Seharusnya BFS tidak memiliki cost, tetapi untuk perhitungan metrics ini diasumsikan 1 cost untuk setiap langkah sehingga path_cost = path_length.
Execution time	Waktu eksekusi pencarian dalam milidetik, diukur menggunakan time.perf_counter()*, dihitung dengan selisih waktu sebelum dan sesudah pencarian.

2.1.3 Algoritma 3: Uniform-Cost Search (UCS)

Referensi: Uniform Cost Search (UCS) in AI - GeeksforGeeks

Formulasi Masalah

- ***State***: posisi kepala (x, y)
- ***Initial state***: posisi kepala saat ini
- ***Goal state***: posisi kepala berada di salah satu apel
- ***Actions***: UP, DOWN, LEFT, RIGHT, dengan urutan sesuai enum
- ***Transition***: (x, y) menjadi (x+dx, y+dy)
- ***Legal move***: di dalam grid dan tidak menabrak badan ular (termasuk lawan)
- ***Path cost***: 1 per langkah, jadi g(n) = jumlah langkah
- ***Obstacle*****:** posisi yang ditempati tubuh ular, tembok, berdasarkan konfigurasi saat pencarian dimulai.



Gambar 3. UCS

Deskripsi Algoritma

Implementasi UCS menggunakan list **agenda** (antrian prioritas) sebagai cara untuk menyimpan pasangan biaya yang telah dikumpulkan dan posisi. Dalam setiap langkah, algoritma memilih keadaan yang memiliki biaya paling rendah/minimal untuk diproses.

Struktur **best** menyimpan biaya terbaik yang ditemukan untuk setiap posisi, sedangkan **parent** menyimpan posisi awal dan arah pergerakan untuk membangun kembali jalur solusi.

Tahapan algoritma adalah sebagai berikut.

1. Mengatur **agenda** dengan posisi awal dan biaya 0.
2. Mengambil ***state*** yang memiliki biaya total paling rendah di **agenda**.
3. Mengabaikan entri yang sudah usang, yaitu entri yang total biayanya sudah tidak lebih baik dibandingkan biaya terbaik yang sudah dicatat.
4. Menghitung status yang lolos dari pemeriksaan sebagai evaluated.
5. Memeriksa apakah ***state*** merupakan goal. Jika ya, pencarian dihentikan.
6. Jika bukan tujuan, state non-root dianggap sebagai **expanded**.
7. Membuat **successor** dengan memeriksa batas papan dan rintangan.
8. Menghitung setiap **successor legal** sebagai **generated**, termasuk **successor** yang posisinya sudah tercatat sebelumnya.
9. Memperbarui **best, parent**, dan **agenda** jika ditemukan biaya yang lebih baik.
10. Jika goal ditemukan, merekonstruksi jalur untuk mendapatkan langkah pertama yang akan dijalankan oleh Snake.
11. Jika tidak ditemukan jalur menuju apel, implementasi memilih gerakan legal/*legal move* secara acak sebagai fallback.

Tabel 4. Definisi Metrics UCS

Metrics	Definisi
Number of food items collected	Jumlah apel yang dimakan Snake.
Success rate	Rasio jumlah apel yang dimakan dengan jumlah apel yang ditetapkan (dalam *setup*)
Generated	Jumlah successor legal yang dihasilkan, termasuk posisi yang sudah tercatat di best
Expanded	Jumlah state yang dikembangkan untuk menghasilkan successor, tidak termasuk initial state
Evaluated	Jumlah state yang diperiksa atau dievaluasi oleh algoritma, sesuai definisi tugas
Path Length	Jumlah gerakan pada jalur solusi yang ditemukan
Path Cost	Total biaya jalur solusi, karena cost=1 untuk setiap langkah, Path Length=Path Cost pada kasus ini.
Execution time	waktu eksekusi pencarian dalam milidetik, diukur menggunakan time.perf_counter()*. Pengukuran ini digunakan untuk mengevaluasi durasi yang dibutuhkan algoritma dalam menentukan langkah. referensi: Benchmark Python with time.perf_counter() – SuperFastPython

2.1.4 Algoritma 4: A* Search

Formulasi Masalah

- ***State***: Kondisi ular pada satu titik pencarian adalah posisi kepala dan seluruh badan. Badan ikut dicatat karena badan bergeser setiap langkah. Posisi apel dianggap tidak berubah selama pencarian (karena posisi apel random dan tidak bergeser saat permainan dimulai).
- ***Initial state***: Kondisi ular saat ini. Kepalanya di snake.body[0] (posisi awal ular secara random), jalur masih kosong, dan biaya g = 0.
- ***Goal state***: Targetnya adalah apel dengan jarak paling dekat ke kepala. Jika ada apel

yang jaraknya sama, dipilih berdasarkan koordinatnya supaya target tidak berganti-ganti.

- ***Actions***: Empat arah diantaranya UP, DOWN, LEFT, RIGHT serta tidak ada gerak diagonal.
- ***Transition***: Satu langkah ke suatu arah dimana kepala maju satu kotak, dan panjang badan tetap kecuali sudah makan apel. Posisi awal, ular belum makan.
- ***Legal move***: Arah yang boleh diambil (UP, DOWN, LEFT, RIGHT) dengan syaratnya tidak boleh berbalik arah, tidak boleh keluar papan, serta tidak boleh menabrak badan sendiri (ekor dikecualikan karena ikut bergerak).
- ***Path cost***: Setiap langkah berbiaya 1 dengan g = jumlah langkah dari awal dan h = jarak ke apel kemudian menghitung f = g + h
- ***Obstacle*:** Dinding dan badan ular sendiri (kecuali ekor)

Deskripsi Algoritma

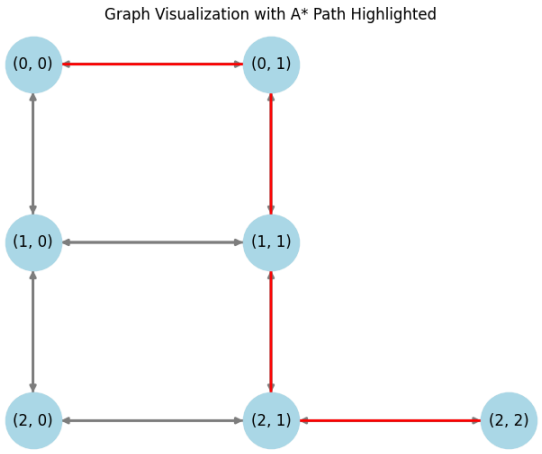
Algoritma ini memilih langkah dengan cara mengira-ngira total jarak (jumlah yang sudah ditempuh dengan sisa yang harus ditempuh) ke apel. Perkiraan itu dihitung dengan rumus f = g + h dengan penjelasan sebagai berikut:

- g adalah jumlah langkah yang sudah ditempuh.
- h adalah perkiraan sisa langkah ke apel, dihitung dengan jarak Manhattan |dx| + |dy|.
- f adalah jumlah keduanya, yaitu perkiraan total langkah.

Cara kerjanya setiap kali ular harus bergerak:

1. Pengecekan di awal, jika tidak ada langkah legal atau tidak ada apel, program langsung mengembalikan langkah legal pertama. Jika ular benar-benar terjebak, program mengembalikan arah sekarang.
2. Tahap persiapan program akan mengumpulkan semua syarat rintangan, lalu memilih apel terdekat sebagai target.
3. Mempersiapkan daftar pencarian dengan daftar ‘frontier’ berisi titik-titik(node) yang ditemukan tetapi belum dijelajahi. Awalnya isinya hanya kondisi ular saat ini. Daftar ‘visited’ mencatat posisi kepala yang sudah dijelajahi.

4. Ulangi sampai selesai:
    - Ambil titik dengan nilai f terkecil dari ‘frontier’. Jika nilainya sama, pilih yang h-nya lebih kecil. Pencarian dilakukan manual dengan looping.
    - Jika posisi kepalanya sudah pernah dilewati, maka akan lewati. Jika belum, catat /simpan ke ‘visited’.
    - Jika ‘visited’ melebihi 5000, pencarian dihentikan, karena di worker.py dibatasi dengan batas waktu 5 detik.
    - Jika kepala sudah sampai di apel, ambil langkah pertama dari jalurnya
    - Jika belum sampai, coba semua arah legal, maka untuk tiap arah, hitung g, h, dan f-nya, lalu masukkan ke ‘frontier’.
    - Jika apel tidak bisa dicapai, maka ular akan memilih arah yang paling luas. Program menghitung luas area kosong di setiap arah dengan flood-fill, lalu memilih area terluas. Jika luasnya sama, dipilih arah yang lebih dekat ke apel. Tujuannya supaya ular tidak terjebak oleh badannya sendiri.



Gambar 4. A* (GeeksforGeeks, Update 2026)

Tabel 5. Definisi Metrics A*

Metrics	Definisi
Number of food items collected	Jumlah apel yang berhasil dikumpulkan ular dalam satu permainan (percobaan). Nilai ini menunjukkan apakah seluruh apel yang diset pada permainan berhasil dihabiskan atau permainan berhenti di tengah jalan.
Success rate	Persentase percobaan di mana algoritma berhasil menemukan

	jalur ke apel target, dihitung dari jumlah pencarian yang berhasil dibagi jumlah seluruh pencarian.
Generated	Jumlah titik (node) yang dibangkitkan selama pencarian, yaitu seluruh titik tetangga yang boleh dilewati dan dimasukkan ke frontier, termasuk yang belum atau tidak pernah diperiksa.
Expanded	Jumlah titik yang diambil dari frontier lalu diperluas, yaitu titik yang tetangganya dibangkitkan untuk diperiksa pada langkah berikutnya.
Evaluated	Jumlah titik yang diperiksa atau dicek apakah sudah mencapai apel (goal test) setelah diambil dari frontier.
Path Length	Jumlah langkah (kotak) pada jalur yang ditempuh ular dari posisi kepala awal sampai apel target, dirata-ratakan per apel.
Path Cost	Total biaya jalur dari posisi awal sampai apel target. Pada permainan ini setiap langkah berbiaya 1, sehingga nilainya mengikuti panjang jalur.
Execution time	Waktu komputasi, dalam milidetik, yang dibutuhkan algoritma untuk menemukan jalur ke apel target, dengan batas waktu maksimum 5 detik.

2.2 Multiplayer Mode

2.2.1 Minimax

Formulasi Masalah

- ***State***: Kondisi ular pada satu titik pencarian adalah posisi kepala dan seluruh badan. Badan ikut dicatat karena badan bergeser setiap langkah. Posisi apel dianggap tidak berubah selama pencarian (karena posisi apel random dan tidak bergeser saat permainan dimulai).
- ***Initial state***: Kondisi ular saat ini. Kepalanya di snake.body[0] (posisi awal ular secara random), jalur masih kosong.
- ***Goal state***: Memilih langkah pertama dengan nilai minimax tertinggi.
- ***Actions***: Empat arah diantaranya UP, DOWN, LEFT, RIGHT serta tidak ada gerak diagonal, kecuali snake tidak bisa mengarah mundur.

- ***Transition***: Satu langkah ke suatu arah dimana kepala maju satu kotak, dan panjang badan tetap kecuali sudah makan apel. Posisi awal, ular belum makan.
- ***Legal move***: Arah yang boleh diambil (UP, DOWN, LEFT, RIGHT) dengan syaratnya tidak boleh berbalik arah, tidak boleh keluar papan, serta tidak boleh menabrak badan sendiri (ekor dikecualikan karena ikut bergerak) dan tidak boleh menabrak badan lawan.
- ***Terminal Test***: Apel habis (pemenang ditentukan dari skor tertinggi, jika skor sama berarti seri), atau pemain tidak punya langkah legal (dianggap kalah).
- ***Utility Function*****:** Nilai terminal untuk menang/kalah/seri, dan fungsi evaluasi heuristik untuk node yang berhenti di batas kedalaman
- ***Obstacle*****:** Dinding, badan ular sendiri (kecuali ekor), dan badan lawan

Deskripsi Algoritma

1. **Asumsi**
    - **Dua pemain.** Pada mode multiplayer, pertandingan hanya antara Minimax dan Alpha-Beta, sehingga selalu ada tepat satu lawan. Jika jumlah lawan bukan satu, strategi memilih langkah legal secara acak.
    - **Gerakan bergantian.** Pada model pencarian, ular bergerak bergiliran (MAX lalu MIN). Pada game sebenarnya kedua ular dapat bergerak bersamaan, sehingga model ini adalah aproksimasi.
    - **Lawan optimal.** MIN diasumsikan selalu memilih langkah yang paling merugikan AI. Asumsi ini wajar karena lawan adalah Alpha-Beta, yang juga mencari langkah optimal.
    - **Tidak ada langkah legal berarti kalah.** Asumsi ini harus konsisten dengan aturan game runner.
    - **Permainan berakhir saat apel habis.** Pemenang ditentukan dari skor tertinggi.
    - **Ekor.** Ekor sendiri boleh dimasuki bila ular tidak tumbuh pada langkah itu. Ekor lawan dianggap tetap terisi (pendekatan konservatif).
    - **Bobot evaluasi.** SCORE_W harus lebih besar dari DIST_W × (rows + cols) agar makan apel selalu lebih bernilai daripada sekadar mendekati apel. WIN harus lebih besar dari SCORE_W × jumlah apel agar nilai menang/kalah selalu mengalahkan heuristik.

Tabel 6. Parameter Minmax

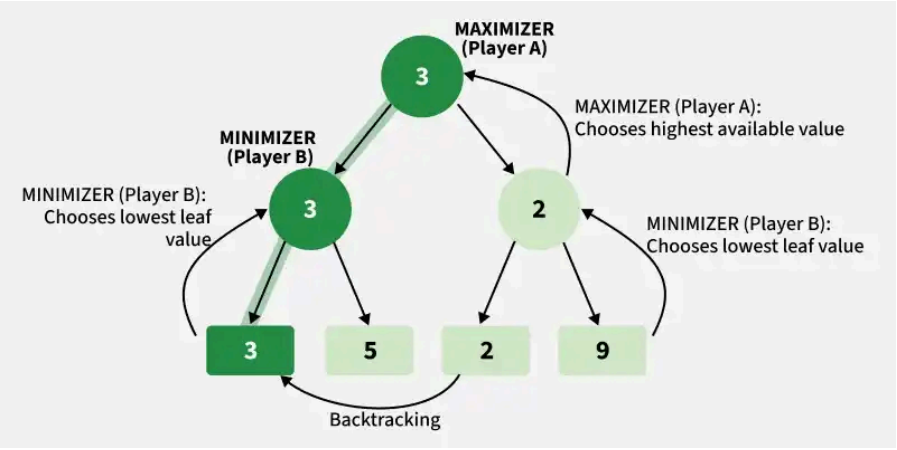
Parameter	Nilai	Fungsi
DEPTH_LIMIT	6	Batas kedalaman pencarian (6 ply)
SCORE_W	1000	Bobot selisih skor
DIST_W	1	Bobot selisih jarak ke apel terdekat
WIN	100000	Nilai menang/kalah mutlak

2. Penjelasan

Minimax dijalankan sebagai pencarian depth-first (DFS): satu cabang ditelusuri sampai batas kedalaman sebelum pindah ke cabang berikutnya. Frontier disimpan sebagai stack (LIFO) yang dikelola otomatis oleh call stack fungsi rekursif minimax. Isinya adalah state yang sudah dibangkitkan tetapi belum dieksplorasi, yaitu saudara-saudara yang belum dikunjungi pada jalur yang sedang ditelusuri.

Pohon permainan terlalu besar untuk dijelajahi sampai akhir permainan, sehingga kedalaman dibatasi 6 ply (AI, lawan, AI, lawan, AI, lawan). Root berada di kedalaman 0. Ada dua jenis node akhir:

- **Node terminal**: permainan benar-benar berakhir (apel habis atau pemain buntu). Dinilai dengan nilai menang/kalah/seri.
- **Node *cutoff***: berhenti karena mencapai DEPTH_LIMIT. Dinilai dengan fungsi evaluasi heuristik untuk menentukan seberapa menguntungkan posisi tersebut bagi AI.



Gambar 5. Minimax (GeeksforGeeks, Update 2026)

Referensi:

Minimax Algorithm in Game Theory | Set 1 (Introduction) - GeeksforGeeks

3. **Langkah-langkah**
   1) **Inisialisasi.** Catat waktu mulai dan nol-kan counter expanded, generated, evaluated.
   2) **Pengecekan awal.** Ambil langkah legal dari snapshot. Jika kosong, kembalikan arah saat ini. Jika lawan tidak tepat satu, pilih langkah legal secara acak.
   3) **Bentuk state awal** dari snapshot.
   4) **Bangkitkan anak root** (giliran MAX): untuk setiap arah valid hitung kepala baru, periksa tabrakan, perbarui tubuh, skor, dan apel. Jika apel habis, tandai terminal dan tentukan pemenang.
   5) **Panggil minimax untuk setiap anak root** (kedalaman 1, giliran MIN):
      i. Jika state terminal, kembalikan nilai terminal.
      ii. Jika kedalaman sama dengan DEPTH_LIMIT, kembalikan nilai evaluasi.
      iii. Bangkitkan anak untuk pemain yang berjalan. Jika tidak ada anak, pemain itu dianggap kalah.
      iv. MAX mengambil nilai maksimum dari anak-anaknya, MIN mengambil nilai minimum.
   6) **Pilih langkah terbaik**. Kumpulkan arah dengan nilai tertinggi. Bila ada beberapa yang sama, pilih acak.
   7) **Fallback.** Jika langkah terpilih kosong atau tidak ada di daftar legal engine, pilih langkah legal secara acak.
   8) **Simpan metrics** dan kembalikan arah terpilih.
4. **Penghitungan Nilai**

   Fungsi evaluasi untuk node cutoff. Jika masih ada apel:

   $Eval = SCORE_W \times (skor_{AI} - skor_{lawan}) + DIST_W \times (d_{lawan} - d_{AI})$

   - d adalah jarak Manhattan dari kepala ke apel terdekat. Jika apel sudah habis, Eval = SCORE_W × selisih skor.

   Contoh: skor AI 2, skor lawan 1, jarak lawan ke apel 5, jarak AI ke apel 3. Eval = $1000 \times (2 - 1) + 1 \times (5 - 3) = 1002$.

   Nilai terminal dan buntu:

Tabel 7. Tabel Kondisi

Kondisi	Rumus
AI menang	WIN − depth + diff
AI kalah	−WIN + depth + diff
Seri	diff (bernilai 0)
MAX buntu	−WIN + depth
MIN buntu	WIN − depth

Suku depth membuat kemenangan yang lebih cepat dan kekalahan yang lebih lambat lebih disukai.

Contoh:

Menang pada kedalaman 2 dengan selisih skor 1 menghasilkan 100000 − 2 + 1 = **99999**. Kalah pada kedalaman 3 dengan selisih skor −1 menghasilkan −100000 + 3 − 1 = **−99998**.

Definisi Metrics

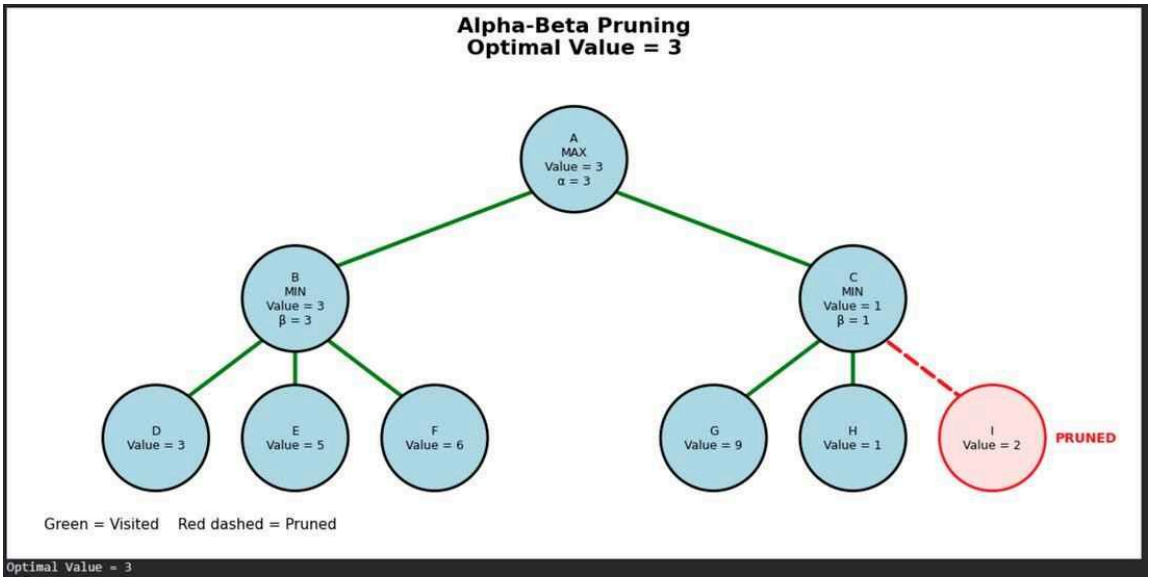
Setiap kali choose_move selesai, hasil pengukuran disimpan di self.last_metrics. Fungsi simpan_metrics dipanggil pada semua jalur keluar (exit) sehingga strukturnya selalu konsisten untuk memudahkan perbandingan.

Tabel 8. Definisi Metrics Minmax

Metrics	**Definisi**
algorithm	Nama algoritma ("Minimax")
snake_id	Id ular yang mengambil keputusan
expanded	Jumlah node internal yang diperluas di dalam rekursi (root tidak dihitung)
generated	Jumlah state anak yang dibangkitkan, termasuk anak root
evaluated	Jumlah node yang dinilai
pruned_branches	Jumlah cabang yang dipangkas
search_depth	Batas kedalaman pencarian
path_len	Panjang jalur langkah yang dikembalikan
cost	Biaya langkah yang dijalankan (1 per langkah), agar total sama dengan jumlah langkah

Metrics	Definisi
Minimax Value	Nilai minimax terbaik di root, hanya untuk analisis dan tidak dijumlahkan
waktu_ms	Waktu komputasi dalam milidetik

2.2.2 Alpha-Beta Pruning



Gambar 6. Alpha-Beta (GeeksforGeeks, Update 2026)

Formulasi Masalah

- ***State***: Kondisi permainan, yaitu posisi kepala dan seluruh badan ular kita, posisi kepala dan seluruh badan ular lawan, daftar apel yang tersisa, dan giliran siapa yang sedang bergerak secara bergiliran (kita atau lawan). Badan ikut dicatat karena badan bergeser setiap langkah.
- ***Initial state***: Kondisi permainan saat ini adalah posisi dan arah ular kita, posisi dan arah ular lawan, serta apel yang ada (posisi awalnya acak). Pencarian dimulai dari giliran kita dengan kedalaman 1 sampai 5, alpha = -tak hingga, dan beta = +tak hingga.
- ***Goal state***: Pencarian berhenti pada tiga kondisi, yang pertama awan tidak punya langkah legal, yang kedua kita tidak punya langkah legal, atau yang ketiga kedalaman pencarian habis. Tujuan akhirnya memilih langkah pertama yang nilainya paling baik.

- ***Actions***: Empat arah, yaitu UP, DOWN, LEFT, RIGHT, tanpa gerak diagonal. Kedua ular memakai pilihan arah yang sama dan bergantian bergerak.
- ***Transition***: Satu langkah ke suatu arah, di mana kepala maju satu kotak. Jika kepala baru jatuh di apel, badan bertambah panjang dan apel itu dihapus dari daftar. Jika tidak, ekor dilepas dan panjang badan tetap. Setelah itu giliran berpindah ke lawan.
- ***Legal move***: Arah yang boleh diambil, dengan syarat tidak berbalik arah, tidak keluar papan, tidak menabrak badan sendiri (ekor dikecualikan karena ikut bergerak), dan tidak menabrak badan ular lawan.
- ***Path cost***: Setiap kondisi diberi nilai evaluasi dari sudut pandang kita: (panjang badan kita − panjang badan lawan) × 100 − jarak ke apel terdekat. Semakin besar nilainya, semakin baik bagi kita. Nilai menang +10.000 dan kalah -10.000 selalu mengalahkan nilai biasa.
- ***Obstacle*:** Dinding, badan ular sendiri (kecuali ekor), dan badan ular lawan.

Deskripsi Algoritma

Alpha-Beta Pruning dipakai untuk permainan dua pemain yang saling berlawanan. Algoritma ini mengibaratkan permainan sebagai pohon kemungkinan: untuk giliran kita, kita akan memilih langkah dengan nilai terbesar, sedangkan pada giliran lawan, lawan akan memilih langkah yang membuat nilai kita terkecil. Cara ini sama dengan Minimax. Bedanya,Alpha-Beta membuang cabang yang sudah pasti tidak akan dipilih, sehingga pencarian lebih cepat tanpa mengubah keputusan akhir (Russell & Norvig, 2021; Knuth & Moore, 1975).

Dua nilai dibawa turun ke setiap cabang:

- Alpha adalah nilai terbaik yang sudah pasti bisa dicapai pihak yang memaksimalkan (kita).
- Beta adalah nilai terbaik yang sudah pasti bisa dicapai pihak yang meminimalkan (lawan).

Cara kerjanya setiap kali ular harus bergerak:

1. Pengecekan di awal dilakukan jika tidak ada langkah legal, program mengembalikan arah sekarang agar tidak error.
2. Selanjutnya mencari lawan dimana program mencari ular lain di papan kemudian, jika tidak ada lawan (mode Single Player), Alpha-Beta tidak

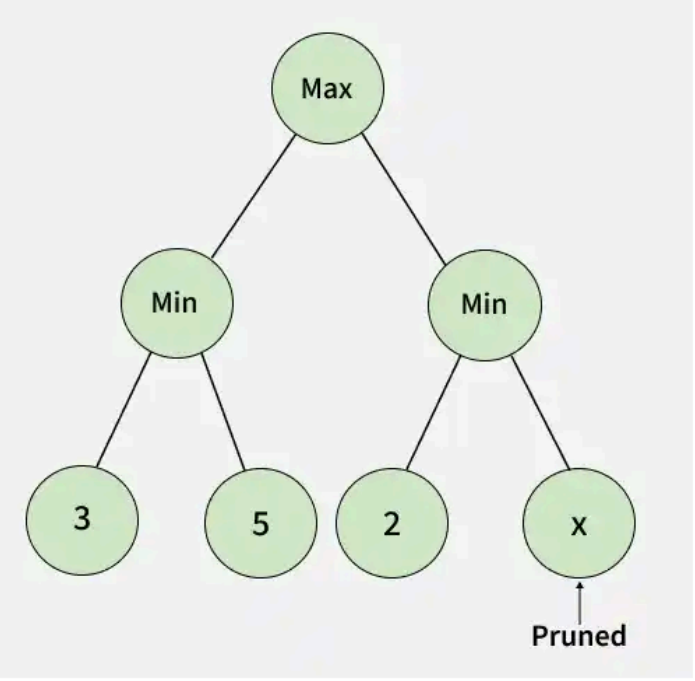
berlaku. Ular langsung memilih langkah yang paling dekat ke apel, dan bila ada yang sama dipilih acak.

3. Menyiapkan fungsi bantu di dalam choose_move, yaitu ‘simulate’ untuk menjalankan satu langkah, legal_from untuk menentukan arah yang boleh diambil, ‘evaluate’ untuk memberi nilai pada suatu kondisi, dan alphabeta untuk pencarian utama.
4. Memeriksa langkah pertama, untuk setiap langkah legal milik kita, program mensimulasikan langkah itu, lalu memanggil alphabeta untuk giliran lawan dengan kedalaman tersisa 5.
5. Pencarian di dalam alphabeta:
    - Jika giliran kita sudah tidak ada lagi langkah legal, maka nilainya -10.000 (kalah). Jika giliran lawan dan lawan tidak punya langkah legal, maka nilainya +10.000 (menang).
    - Jika kedalaman habis, kondisi dinilai dengan ‘evaluate’.
    - Giliran kita: akan coba setiap arah legal, lalu mengambil nilai terbesar, dan perbarui alpha.
    - Giliran lawan: akan coba setiap arah legal, lalu ambil nilai terkecil, dan perbarui beta.
    - Pemangkasan akan terjadi ketika alpha >= beta, sisa cabang di titik itu tidak akan diperiksa lagi. Cabang di atas sudah memiliki pilihan lain yang pasti lebih baik, jadi cabang tersebut tidak akan pernah dipilih sehingga terjadi *pruning*.

Tabel 9. Definisi Metrics Alpha-Beta

Metrics	Definisi
Algoritma	Algoritma pencarian yang diuji, yaitu Alpha-Beta Pruning, yaitu pengembangan Minimax yang membuang cabang yang pasti tidak akan dipilih tanpa mengubah keputusan akhir.
Depth	Kedalaman maksimum pencarian, yaitu berapa langkah ke depan (giliran kita dan lawan bergantian) yang disimulasikan sebelum kondisi dinilai dengan fungsi evaluate.
Win Rate (%)	Persentase permainan yang dimenangkan ular kita dari seluruh percobaan, dihitung dari jumlah kemenangan dibagi jumlah

	percobaan. Menang terjadi ketika lawan tidak punya langkah legal.
Avg Score	Rata-rata skor akhir yang diperoleh ular kita per percobaan pada kedalaman tertentu.
States Generated	Jumlah kondisi permainan yang dibangkitkan lewat simulasi langkah (simulate) selama pencarian, termasuk kondisi dari cabang yang kemudian dipangkas.
States Evaluated	Jumlah kondisi yang benar-benar diberi nilai oleh fungsi evaluate, yaitu kondisi yang mencapai batas kedalaman. Kondisi pada cabang yang dipangkas tidak ikut dinilai.
Pruned Branches	Jumlah cabang yang dipangkas (tidak diperiksa lagi) karena kondisi alpha ≥ beta terpenuhi. Ini menunjukkan seberapa banyak pekerjaan pencarian yang berhasil dihemat.
Exec Time / Move (ms)	Rata-rata waktu komputasi, dalam milidetik, yang dibutuhkan algoritma untuk memilih satu langkah.

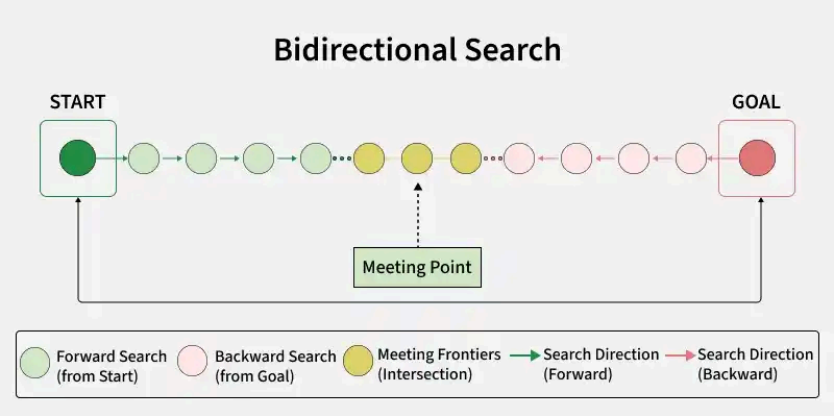


Gambar 7. Alpha-Beta Pruning(GeeksforGeeks, Update 2026)

6. Selanjutnya memilih langkah. Di akar paling atas, program memilih langkah dengan nilai terbaik. alpha dibawa terus antar cabang akar supaya cabang yang

diperiksa belakangan ikut terpangkas lebih awal. Jika beberapa langkah sama baiknya, maka dipilih acak agar tidak mudah ditebak.

2.3 Algoritma Bonus - Algoritma: Bidirectional Search



Gambar 8. Bidirectional Search (GeeksforGeeks, Update 2026)

Formulasi Masalah

- ***State***: Posisi kepala ular dalam x, y; posisi apel dalam x, y yang tidak berubah-ubah sejak pertama kali menjalankan game.
- ***Initial state***: Posisi kepala ular saat awal game yaitu snake.body[0] dan posisi apel pertama yaitu snapshot.apples[0], movement masih kosong, queue_snake berisi posisi kepala ular awal, queue_apple berisi posisi apel, serta visited_snake dan visited_apple masing-masing berisi posisi awal pencarian.
- ***Goal state***: Pencarian dari sisi kepala ular bertemu dengan pencarian dari sisi apel.
- ***Actions***: Ular dapat bergerak ke arah UP, DOWN, LEFT, RIGHT.
- ***Transition***: Transisi terjadi ketika posisi pencarian dari kepala ular dan posisi pencarian dari apel bergerak satu kotak setelahnya sesuai arah yang diperbolehkan. Transisi posisi x, y adalah x+dx, y+dy.
- ***Legal move***: Arah yang diperbolehkan pada ular yaitu UP, DOWN, LEFT, RIGHT, posisi ular x tidak boleh lebih dari jumlah kolom, posisi ular y tidak boleh lebih dari jumlah baris, tidak boleh bergerak ke arah badan ular. Pada pencarian dari sisi apel, algoritma hanya memeriksa posisi yang sudah visited dan batas papan.
- ***Path cost***: Untuk Bidirectional Search secara teori tidak ada path cost, tetapi pada algoritma yang dibuat diasumsikan adalah 1 untuk setiap langkah. g(n) = jumlah langkah dari initial state hingga final state.

❖ ***Obstacle*:** Dinding papan dan badan ular.

Deskripsi Algoritma

Bidirectional Search merupakan algoritma searching yang melakukan *searching* dari dua arah, yaitu dari *initial state* (posisi awal kepala ular) menuju *goal state* (apel) dan sebaliknya. *Searching* dilakukan bersama hingga bertemu di posisi yang sama. Bidirectional Search menggunakan queue (antrian) yaitu queue_snake untuk menyimpan posisi kepala ular beserta movement yang sudah diperiksa, dan queue_apple untuk menyimpan posisi dalam pencarian dari sisi apel. Kedua queue menggunakan First-In First-Out sehingga state yang diproses lebih dulu adalah state yang berada di posisi paling depan queue. Supaya state yang sudah diproses ini tidak kembali masuk ke dalam queue dan menyebabkan *infinite loop*, ada visited_snake dan visited_apple untuk menyimpan posisi yang sudah dikunjungi. apple_depth menyimpan jarak setiap posisi yang ditemukan oleh pencarian dari apel terhadap posisi apel awal (GeeksforGeeks, Update 2026).

Tahapan algoritma adalah sebagai berikut.

1. Inisialisasi *searching*: posisi kepala ular dan posisi apel menjadi titik awal pencarian dari dua arah. queue_snake diisi dengan posisi kepala ular awal dan movement kosong, sedangkan queue_apple diisi dengan posisi dari apel. Posisi kepala ular dimasukkan ke dalam visited_snake, posisi apel dimasukkan ke dalam visited_apple, serta apple_depth untuk posisi apel diinisialisasi dengan nilai 0.
2. Ambil *state* dari queue_snake: menggunakan queue_snake.pop(0) untuk mengambil dan menghapus state paling depan pada queue pencarian dari kepala ular.
3. Menghitung expanded state jika movement tidak kosong. *Initial state* tidak dihitung sebagai expanded karena belum ada movement.
4. Memeriksa apakah posisi kepala ular sudah terdapat dalam visited_apple. Jika sudah ada, kedua pencarian telah bertemu. Panjang jalur dihitung dari jumlah gerakan pada movement ditambah jarak posisi pertemuan terhadap apel yang disimpan dalam apple_depth.
5. Jika kedua pencarian belum bertemu, Bidirectional Search memeriksa setiap arah dari posisi kepala ular. Langkah arah yang tidak termasuk dalam legal akan dilewati. Jika posisi baru masih berada di dalam papan, generated state akan bertambah. Jika posisi

baru belum dikunjungi oleh pencarian ular, posisi tersebut dimasukkan ke dalam visited_snake dan state baru dimasukkan ke dalam queue_snake beserta movement-nya.

6. Ambil *state* dari queue_apple: menggunakan queue_apple.pop(0) untuk mengambil dan menghapus posisi terdepan pada queue pencarian dari apel.
7. Memeriksa apakah posisi pencarian dari sisi apel sudah terdapat dalam visited_snake. Jika ada, algoritma memeriksa apakah terdapat posisi yang sama dalam queue_snake. Jika ada, gerakan pertama dari jalur tersebut disimpan sebagai solusi dan panjang jalur dihitung berdasarkan jumlah gerakan dari kepala ular ditambah jarak posisi pertemuan terhadap apel.
8. Searching berhasil ketika posisi pencarian apel sama dengan posisi pencarian dari kepala ular. Jika kedua pencarian belum bertemu, Bidirectional Search akan memeriksa tiap arah dari posisi pencarian apel. Jika posisi baru masih berada di dalam papan, generated state akan bertambah. Jika posisi tersebut belum dikunjungi oleh pencarian apel, posisi dimasukkan ke dalam visited_apple, jaraknya terhadap apel dihitung dan disimpan dalam apple_depth, lalu posisi baru dimasukkan ke dalam queue_apple.
9. Menentukan gerakan atau solusi untuk ular dengan mengembalikan arah pertama dari movement.

Tabel 10. Definisi Metrics Bidirectional Search

Metrics	Definisi
Number of food items collected	Jumlah apel yang dimakan Snake.
Success rate	Rasio jumlah apel yang dimakan dengan jumlah apel yang ditetapkan dalam setup.
Generated	Jumlah successor posisi dari kedua pencarian yang berada di dalam batas papan, termasuk posisi yang sudah pernah dikunjungi.
Expanded	Jumlah state dari kedua pencarian selain initial state yang diproses sebelum proses searching selesai untuk menghasilkan successor. Pada pencarian ular, state dihitung jika movement tidak kosong. Pada pencarian apel, state dihitung jika apple_depth lebih dari 0.

Evaluated	Jumlah state yang diperiksa oleh algoritma. Evaluated sejumlah expanded karena initial state tidak dihitung dan state lainnya dihitung sebagai expanded.
Path Length	Jumlah gerakan pada jalur solusi yang ditemukan ditambah jarak posisi pertemuan terhadap apel.
Path Cost	Seharusnya Bidirectional Search tidak memiliki cost, tetapi untuk perhitungan metrics ini diasumsikan 1 cost untuk setiap langkah sehingga path_cost = path_length.
Execution time	Waktu eksekusi pencarian dalam milidetik, diukur menggunakan time.perf_counter()*, dihitung dengan selisih waktu sebelum dan sesudah pencarian.

BAB 3

Eksperimen dan Evaluasi

3.1 Evaluasi Single Player Mode

3.1.1 Hasil Pengujian Single Player

Tabel 11. Tabel Pengujian Single Player

Algoritma	Makanan Terkumpul (Avg)	*Success Rate* (%)	*States Expanded*	*States Generated*	*Path Length*	*Path Cost*	*Exec Time* (ms)
DFS	2.8	56	17932.6	67365.2	14480.6	14480.6	345.286
BFS	5	100	10182.8	21900.8	395.8	395.8	218.25
UCS	5	100	4408	16831.2	339	339.00	63.82
A*	5	100	6.77	17.57	5.77	5.77	0.906
Bidirectional Search	5	100	5792.4	22593.6	657.6	657.6	71.182

3.1.2 Analisis dan Pembahasan *Single Player*

- **DFS vs BFS:**

  [Perbandingan kelengkapan, optimalitas, dan penggunaan memori]

  Dalam hal *completeness*, dengan jumlah pergerakan yang dibatasi, BFS lebih lengkap dibandingkan DFS karena BFS mencari dari tingkat simpul terdekat terlebih dahulu sehingga dapat menemukan solusi lebih cepat. DFS mencari dengan menelusuri suatu cabang hingga ke tingkat terdalamnya sehingga berisiko tidak menemukan solusi bila ternyata solusi tersedia di cabang lainnya. Hal ini terlihat dari hasil pengujian, yaitu BFS memperoleh success rate sebesar 100%, sedangkan DFS hanya 56%. Dalam hal optimalitas, BFS lebih optimal karena dapat menemukan movement(path) dengan langkah yang lebih sedikit dibandingkan DFS. DFS tidak menjamin mendapatkan jalur terpendek karena menelusuri cabang hingga dalam terlebih dahulu. Hal ini terlihat dari

hasil pengujian, yaitu BFS memperoleh path length sebesar 395.8, sedangkan DFS 14480.6. Dalam hal penggunaan memori, berdasarkan hasil pengujian, DFS menghasilkan states generated sebesar 67365.2, sedangkan BFS hanya 21900.8. Hal ini menunjukkan bahwa DFS menghasilkan sekitar 3.08 kali lebih banyak state dibandingkan BFS. Dengan demikian, berdasarkan jumlah state yang dihasilkan, BFS lebih efisien dalam proses pencarian.

- **BFS vs UCS:**

  [Perbandingan jumlah langkah dan biaya pergerakan]

  Berdasarkan hasil pengujian, dalam hal jumlah langkah dan biaya pergerakan, UCS lebih baik daripada BFS. BFS memiliki path length dan cost sebesar 395.8, sedangkan UCS hanya 339. Kedua algoritma tersebut memiliki success rate sama-sama 100%, tetapi UCS memiliki states expanded dan states generated lebih sedikit dibandingkan BFS yaitu 4408 vs. 10182.8 states expanded dan 16831.2 vs 21900.8 states generated.

- **A* vs Bidirectional:**

  [Perbandingan kecepatan pencarian vs optimalitas jalur]

  Keduanya sama-sama berhasil mengumpulkan 5 apel di semua percobaan. Bedanya ada di seberapa cepat dan hemat cara kerjanya. Kecepatan. A* jauh lebih cepat. Waktu rata-ratanya hanya sekitar 0,9 ms, sedangkan Bidirectional sekitar 71 ms, atau kira-kira 79 kali lebih lama. Jumlah state yang diperiksa juga sangat berbeda: A* hanya sekitar 7, sedangkan Bidirectional sekitar 5.800. Alasannya, A* punya "petunjuk arah" berupa perkiraan jarak ke apel, jadi ia langsung menuju ke sana. Bidirectional tidak punya petunjuk, jadi ia mencari ke segala arah dari dua sisi sampai kedua pencarian bertemu. Kualitas jalur. Secara teori, keduanya sama-sama bisa menemukan jalur terpendek. Jadi dalam hal ini tidak ada pemenang yang jelas. Angka panjang jalur dari pengujian (5,8 untuk A* dan 657,6 untuk Bidirectional) tidak bisa dibandingkan langsung, karena A* dihitung per apel, sedangkan Bidirectional kemungkinan dihitung per permainan. Kesimpulan. Di permainan ini, A* lebih unggul karena jauh lebih cepat dan hemat, tanpa mengorbankan kualitas jalur. Bidirectional Search lebih baik daripada pencarian satu arah biasa, tetapi tanpa petunjuk arah tetap jauh lebih boros daripada A*.

- **Pengaruh Fungsi Heuristik pada A*:**

  Penggunaan heuristik membuat algoritma A* memiliki arah yang jelas menuju apel target, sehingga jumlah state yang diperluas dan dibangkitkan jauh lebih sedikit dibandingkankan pencarian tanpa heuristik yang menyebar ke segala arah seperti contoh pada BFS dan DFS. Setiap state dinilai menggunakan fungsi f = g + h, sehingga state dengan kepala ular yang lebih dekat ke apel akan diproses lebih dahulu. Heuristik ini bersifat admissible karena jarak tidak pernah melebihi jumlah langkah sebenarnya, sehingga jalur yang ditemukan tetap optimal. Namun, heuristik ini mengabaikan rintangan seperti dinding dan badan ular, sehingga apabila jalur lurus menuju apel terhalang, A* tetap perlu memperluas banyak state sebelum menemukan jalur alternatif.

- **Kualitas Solusi vs Biaya Komputasi:**

  [Trade-off kecepatan vs optimalitas]

  Hasil pengujian menunjukkan bahwa solusi yang bagus tidak selalu butuh komputasi yang besar. DFS adalah contoh terburuk. Ia hanya berhasil 56% dari percobaan, jalurnya sangat panjang (14.480 langkah), dan membangkitkan banyak state (67.365). Sebagai perbandingan, BFS berhasil 100% dengan jalur hanya 395 langkah dan 21.900 state. Jadi DFS kalah dari BFS, baik dari segi kualitas maupun biaya. UCS lebih baik lagi daripada BFS. Jalurnya lebih pendek (339 dibanding 395,8) dan prosesnya lebih ringan: state yang diperiksa sekitar 57% lebih sedikit dan state yang dibangkitkan sekitar 23% lebih sedikit. A* paling efisien. Per apel, ia hanya membangkitkan rata-rata 17,57 state dengan jalur 5,77 langkah, dan waktu eksekusi sekitar 0,9 ms. Heuristik Manhattan membuatnya langsung menuju apel tanpa menyebar ke segala arah, dan jalurnya tetap optimal. Kesimpulannya, di permainan ini tidak ada trade-off tajam antara kecepatan dan kualitas. Algoritma yang menjamin solusi optimal (BFS, UCS, A*) justru lebih hemat daripada DFS. Perbedaan baru terlihat di antara ketiganya: BFS paling boros, UCS lebih hemat, dan A* paling hemat.

- **Analisis Kasus Kegagalan:**

  [Penjelasan kondisi saat ular terjebak/kalah]

  - DFS memiliki success rate hanya 56% karena algoritma tersebut menelusuri satu cabang sedalam mungkin terlebih dahulu sebelum berpindah cabang. Dengan begitu,

dengan adanya batas maksimum 200 move yang diperbolehkan dalam game, DFS sering kehabisan langkah untuk mengambil apel-apel yang ada. Hal ini terlihat dari path lengths DFS yang mencapai 14480.6.

- Untuk algoritma A* bisa terjebak atau kalah dalam beberapa kondisi. Pertama, jika semua arah sudah buntu (semua menabrak dinding atau badan), ular tidak punya pilihan dan pasti akan tabrakan. Kedua, ular memilih apel yang paling dekat jaraknya, padahal apel itu belum tentu bisa dicapai karena jalannya bisa tertutup badan sendiri atau bisa jadi ular lawan. Seiandainya seperti itu, A* akan terus mencari jalan sampai kehabisan kemungkinan atau mencapai batas 5000 state, lalu menyerah. Ketiga, saat menyerah, ular akan memilih langkah secara acak tanpa berpikir ke depan, jadi bisa masuk ke jalan buntu. Keempat, A* hanya memikirkan cara sampai ke apel dan tidak memeriksa apakah setelah makan ular masih punya ruang untuk bergerak. Makin panjang badan ular, makin besar risiko terkurung oleh badannya sendiri. Kelima, ular lawan dianggap diam di tempat, padahal mereka terus bergerak, sehingga jalur yang tadinya aman bisa tiba-tiba tertutup atau terjadi tabrakan kepala. Terakhir, kalau tidak ada apel di papan, ular hanya mengambil langkah legal pertama tanpa rencana.

3.2 Evaluasi *Multiplayer Mode*

3.2.1 Hasil Pengujian *Multiplayer*

Tabel 12. Tabel Pengujian Multi Player

Algoritma	Depth	Win Rate (%)	Avg Score	States Generated	States Evaluated	Pruned Branches	Exec Time / Move (ms)
Minimax	d=1	80%	2,8	67,2	67,2	0	3,70
Minimax	d=2	100%	3,2	381	283,2	0	10.11
Minimax	d=3	60%	2,4	1027	700,6	0	14.29
Minimax	d=4	80%	3	2537,8	1681	0	42,09
Minimax	d=5	100%	3	7365,4	4819,2	0	104,25
Minimax	d=6	60%	2,6	27245,8	17763,6	0	327,60

Algoritma	Depth	Win Rate (%)	Avg Score	States Generated	States Evaluated	Pruned Branches	Exec Time / Move (ms)
Alpha-Beta	d=1	60%	3	48	48	0	4.70
Alpha-Beta	d=2	40%	2	198	136	25	14.98
Alpha-Beta	d=3	40%	2	396	245	71	27.39
Alpha-Beta	d=4	80%	4	1935	1064	448	116.19
Alpha-Beta	d=5	40%	2	1570	870	386	103.39
Alpha-Beta	d=6						

3.2.2 Analisis dan Pembahasan *Multiplayer*

- **Minimax vs Alpha-Beta Pruning:**
    - Alpha-Beta "membuang" cabang yang pasti tidak akan dipilih, jadi komputer tidak perlu memeriksanya. Hasilnya, state yang dievaluasi lebih sedikit daripada yang dibangkitkan. Di d=1 belum ada cabang yang dibuang (0), jadi state generated dan evaluated sama, yaitu 48, dan hasilnya mirip Minimax biasa. Mulai d=2, pemangkasan berjalan: 25 cabang dibuang di d=2, lalu 71 di d=3, 448 di d=4, dan 386 di d=5. Akibatnya, rasio state yang dievaluasi turun dari 68,7% (d=2) ke 61,9% (d=3), lalu 55,0% (d=4) dan 55,4% (d=5), artinya makin dalam pencarian, makin banyak pekerjaan yang dihemat.
    - Pada Minimax, state yang dibangkitkan naik sekitar 2,5 sampai 3,7 kali setiap depth bertambah satu (d=2 ke d=3: 2,7 kali; d=3 ke d=4: 2,5 kali; d=4 ke d=5: 2,9 kali; d=5 ke d=6: 3,7 kali), dan waktu per langkah naik dari 3,70 ms (d=1) menjadi 327,60 ms (d=6), sekitar 89 kali. Lonjakan dari d=1 ke d=2 (5,7 kali) lebih besar

karena pada d=1 hanya anak root yang dibangkitkan. State yang dievaluasi mengikuti pola yang sama: 67,2 (d=1), 283,2 (d=2), 700,6 (d=3), 1681 (d=4), 4819,2 (d=5), dan 17763,6 (d=6). Karena Minimax tidak memangkas apa pun, rasio state yang dievaluasi terhadap yang dibangkitkan hanya ditentukan bentuk pohon, yaitu turun lalu stabil di sekitar 65%: 74,3% (d=2), 68,2% (d=3), 66,2% (d=4), 65,4% (d=5), dan 65,2% (d=6). Artinya sekitar dua pertiga node yang dibangkitkan adalah daun, dan sisanya node internal yang hanya diperluas. Waktu per langkah naik 2,7 kali (d=1 ke d=2), 1,4 kali (d=2 ke d=3), 2,9 kali (d=3 ke d=4), 2,5 kali (d=4 ke d=5), dan 3,1 kali (d=5 ke d=6). Kenaikan d=2 ke d=3 yang kecil kemungkinan berasal dari variasi posisi permainan, tetapi secara keseluruhan waktu tetap mengikuti pertumbuhan eksponensial.

- **Pengaruh Search Depth:** Semakin dalam depth, beban komputasi akan makin berat. Alasannya adalah karena setiap tambahan depth berarti komputer harus memikirkan satu langkah lebih jauh ke depan, dan di tiap langkah ada banyak pilihan gerakan yang harus dicek. Jumlah kemungkinan ini berlipat ganda, bukan hanya bertambah sedikit.

Tabel 13. Tabel Pengujian Multi Player

Depth	Generated (Alpha-Beta / Minimax)	Evaluated (Alpha-Beta / Minimax)
d=2	198 / 381 (52%)	136 / 283,2 (48%)
d=3	396 / 1027 (39%)	245 / 700,6 (35%)
d=4	1935 / 2537,8 (76%)	1064 / 1681 (63%)
d=5	1570 / 7365,4 (21%)	870 / 4819,2 (18%)

  - Terlihat dari state yang dibangkitkan, naik dari 48 (d=1) ke 198, 396, lalu 1935 (d=4), dan waktu per langkah ikut naik dari 4,70 ms ke 116,19 ms. Di d=5 angkanya turun sedikit (1570 state, 103,39 ms), kemungkinan karena pemangkasan lebih efektif atau kondisi permainannya berbeda. Soal hasil, win rate tidak naik terus: 3/5

di d=1, 2/5 di d=2 dan d=3, 4/5 di d=4, lalu 2/5 di d=5. Karena tiap depth hanya diuji 5 kali, belum bisa dipastikan depth yang lebih besar selalu lebih baik. Yang jelas terlihat, d=4 hasilnya paling bagus tapi paling berat, sedangkan d=1 sampai d=3 lebih ringan namun win rate-nya lebih rendah.

- Minimax mencatat 80% (d=1), 100% (d=2), 60% (d=3), 80% (d=4), 100% (d=5), dan 60% (d=6), sedangkan Alpha-Beta mencatat 60% (d=1), 40% (d=2 dan d=3), 80% (d=4), dan 40% (d=5). Karena tiap depth hanya diuji 5 kali, selisih 20 poin persentase setara satu pertandingan, sehingga belum bisa dipastikan bahwa depth yang lebih besar selalu lebih baik. Rata-rata skor Minimax juga relatif datar (2,4 sampai 3,2). Pada Minimax, d=6 tidak lebih baik daripada d=2 padahal waktu komputasi sekitar 32 kali lebih besar (327,60 ms lawan 10,11 ms). Jika hanya melihat Minimax, titik terbaik pada data ini adalah d=2: win rate 100%, skor rata-rata tertinggi (3,2), dan hanya 10,11 ms per langkah. Depth d=5 juga mencapai win rate 100% tetapi sekitar 10 kali lebih lambat (104,25 ms), sedangkan d=6 turun ke 60% dengan waktu tertinggi (327,60 ms). Tambahan **depth tidak signifikan** menjamin hasil lebih baik (dalam hal win rate). Kemungkinan penyebabnya adalah fungsi evaluasi yang sama pada semua depth, model gerak bergantian yang hanya aproksimasi dari game sebenarnya, dan sampel yang kecil (5 pertandingan per depth).

BAB 4

Kesimpulan

[Ringkasan temuan utama dari evaluasi Single Player & Multiplayer serta rekomendasi algoritma terbaik]

4.1 Kesimpulan dan Rekomendasi Algoritma *Single Player*

Berdasarkan hasil eksperimen pengujian, setiap algoritma memiliki kemampuan yang berbeda dalam menyelesaikan permainan Snake. DFS menunjukkan kinerja paling rendah dengan success rate hanya 56%, rata-rata makanan terkumpul hanya 2,8, serta path length besar mencapai 14.480,6. Hal ini disebabkan oleh strategi DFS yang menelusuri satu cabang sedalam mungkin sebelum menelusuri cabang lainnya. BFS, UCS, dan Bidirectional Search berhasil mencapai success rate 100% dengan rata-rata 5 makanan terkumpul. Namun, UCS menghasilkan path length terpendek di antara ketiganya, yaitu 339, dengan 4408 states expanded dan waktu eksekusi 63.82 ms. Sementara itu, BFS membutuhkan waktu 218.25 ms dengan path length 395.8, sedangkan Bidirectional Search membutuhkan 71.182 ms dengan path length 657.6.

A* menjadi algoritma dengan kemampuan terbaik berdasarkan data pengujian, dengan waktu eksekusi hanya 0.906 ms, path length hanya 5.77, serta jumlah states expanded dan states generated masing-masing hanya 6.77 dan 17.57. Hasil ini menunjukkan bahwa penggunaan heuristik membantu algoritma mengarahkan pencarian menuju apel secara lebih efisien. A* juga memperoleh success rate 100% dan rata-rata 5 makanan terkumpul. Dengan demikian, A* merupakan rekomendasi algoritma terbaik untuk mode Single Player karena berhasil mengumpulkan makanan secara lengkap dengan biaya komputasi yang sangat rendah dan jalur yang pendek.

4.2 Kesimpulan dan Rekomendasi Algoritma *Multiplayer*

a. Depth dan beban komputasi.

   Semakin dalam pencarian, jumlah state dan waktu per langkah naik tajam. Pada Minimax, state naik sekitar 2,5 sampai 3,7 kali setiap depth bertambah satu, dan waktu per langkah naik dari 3,70 ms (d=1) menjadi 327,60 ms (d=6).

b. Efek pruning.

   Alpha-Beta membangkitkan dan mengevaluasi lebih sedikit state daripada Minimax pada depth yang sama (d=2 sampai d=5), dan rasio state yang dievaluasi terhadap yang

dibangkitkan lebih rendah, yaitu 55% sampai 69% lawan 65% sampai 74%. Penghematan ini belum selalu terlihat pada waktu per langkah karena satuan datanya berbeda (total per pertandingan lawan rata-rata per langkah) dan implementasi keduanya tidak identik.

c. Hasil permainan.

   Depth yang lebih besar tidak otomatis menang lebih sering. Pada percobaan ini Minimax sebagai Player 1 menang lebih sering (rata-rata 84% pada d=1 sampai d=5) daripada Alpha-Beta sebagai Player 1 (52%), tetapi sampelnya hanya 5 pertandingan per depth dan fungsi evaluasi keduanya berbeda, sehingga hasil ini belum bisa dianggap pasti.

d. Secara ringkas, Alpha-Beta lebih hemat komputasi, sedangkan kualitas bermain pada percobaan ini lebih ditentukan oleh fungsi evaluasi dan jumlah sampel daripada oleh pruning itu sendiri. Pengujian dengan skala yang lebih besar (depth, dimensi grid, jumlah apel dan jumlah percobaan) dan penyamaan fungsi evaluasi kedua algoritma dapat dijadikan langkah lanjutan.

Daftar Referensi

GeeksforGeeks. (Update 2026). Depth First Search (DFS) for Artificial Intelligence. https://www.geeksforgeeks.org/artificial-intelligence/depth-first-search-dfs-for-artificial-intelligence/

GeeksforGeeks. (Update 2026). Breadth First Search (BFS) for Artificial Intelligence. https://www.geeksforgeeks.org/artificial-intelligence/breadth-first-search-bfs-for-artificial-intelligence/

GeeksforGeeks. (Update 2026). Bidirectional Search in AI. https://www.geeksforgeeks.org/artificial-intelligence/bidirectional-search-in-ai/

GeeksforGeeks. (Update 2026). State Space Search in AI. https://www.geeksforgeeks.org/artificial-intelligence/state-space-search-in-ai/

GeeksforGeeks. (Update 2026). Bidirectional Search in AI. https://www.geeksforgeeks.org/artificial-intelligence/a-algorithm-and-its-heuristic-search-strategy-in-artificial-intelligence/

GeeksforGeeks. (Update 2026). Bidirectional Search in AI. https://www.geeksforgeeks.org/artificial-intelligence/alpha-beta-pruning-in-adversarial-search-algorithms/

Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4), 293–326.

Lembar Lampiran

- **Justifikasi Modifikasi Framework:** [Penjelasan jika ada perubahan di luar fungsi yang ditentukan]

  Penambahan pencatatan metrics (generated, expanded, evaluated, path length, path cost, execution time) pada file metrics.py yang isinya menjalankan fungsi capture metrics-metrics yang dihitung di strategies.py. Dari metrics.py hasilnya disimpan di metrics.txt. Kemudian pada file strategies.py ada perubahan dengan menambahkan validasi pada pilihan single player dan multiplayer. Selain itu, ada penambahan algoritma baru ke daftar strategi, yakni Bidirectional Search.

Lampiran

Lampiran Depth-First Search (DFS)

Run ke-	Makanan Terkumpul	Success Rate (%)	States Expanded	States Generated	Path Length	Path Cost	Exec Time (ms)
1	4	80	15437	57673	9855	9855	308.70
2	4	80	15922	59847	11996	11996	330.60
3	2	40	17477	66000	16426	16426	318.27
4	2	40	22407	84279	18712	18712	441.75
5	2	40	18420	69027	15414	15414	327.11
Average (Avg)	**2.8**	**56**	**17932.6**	**67365.2**	**14480.6**	**14480.6**	**345.286**

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 22:55:37
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Depth-First Search

--- HASIL MATCH ---
Alasan Selesai: Move limit reached.
Skor AI    : 4 (Langkah: 200)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 4
Success Rate                   : 80.0%
Number of States Expanded      : 15437
Number of States Generated     : 57673
Total Path Length              : 9855
Total Path Cost                : 9855.00
Total Execution Time           : 308.70 ms
----------------------------
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 22:56:35
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Depth-First Search

--- HASIL MATCH ---
Alasan Selesai: Move limit reached.
Skor AI    : 4 (Langkah: 200)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 4
Success Rate                   : 80.0%
Number of States Expanded      : 15922
Number of States Generated     : 59847
Total Path Length              : 11996
Total Path Cost                : 11996.00
Total Execution Time           : 330.60 ms
----------------------------
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 22:58:14
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Depth-First Search

--- HASIL MATCH ---
Alasan Selesai: Move limit reached.
Skor AI   : 2 (Langkah: 200)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 2
Success Rate                   : 40.0%
Number of States Expanded      : 17477
Number of States Generated     : 66000
Total Path Length              : 16426
Total Path Cost                : 16426.00
Total Execution Time           : 318.27 ms
----------------------------
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 23:01:18
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Depth-First Search

--- HASIL MATCH ---
Alasan Selesai: Move limit reached.
Skor AI   : 2 (Langkah: 200)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 2
Success Rate                   : 40.0%
Number of States Expanded      : 22407
Number of States Generated     : 84279
Total Path Length              : 18712
Total Path Cost                : 18712.00
Total Execution Time           : 441.75 ms
----------------------------
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 23:04:40
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Depth-First Search

--- HASIL MATCH ---
Alasan Selesai: Move limit reached.
Skor AI   : 2 (Langkah: 200)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 2
Success Rate                   : 40.0%
Number of States Expanded      : 18420
Number of States Generated     : 69027
Total Path Length              : 15414
Total Path Cost                : 15414.00
Total Execution Time           : 327.11 ms
----------------------------
```

Lampiran Breadth-First Search (BFS)

Run ke-	Makanan Terkumpul	Success Rate (%)	States Expanded	States Generated	Path Length	Path Cost	Exec Time (ms)
1	5	100	6383	18373	367	367	96.83
2	5	100	26318	38527	699	699	673.08
3	5	100	11629	25437	516	516	224.83
4	5	100	1922	5621	158	158	26.28
5	5	100	4662	11546	239	239	70.23
Average (Avg)	**5**	**100**	**10182.8**	**21900.8**	**395.8**	**395.8**	**218.25**

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 23:20:25
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Breadth-First Search

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI    : 5 (Langkah: 56)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 5
Success Rate                   : 100.0%
Number of States Expanded      : 6383
Number of States Generated     : 18373
Total Path Length              : 367
Total Path Cost                : 367.00
Total Execution Time           : 96.83 ms
----------------------------
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 23:21:01
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Breadth-First Search

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI    : 5 (Langkah: 61)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 5
Success Rate                   : 100.0%
Number of States Expanded      : 26318
Number of States Generated     : 38527
Total Path Length              : 699
Total Path Cost                : 699.00
Total Execution Time           : 673.08 ms
----------------------------
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 23:21:19
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Breadth-First Search

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 62)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 5
Success Rate                   : 100.0%
Number of States Expanded      : 11629
Number of States Generated     : 25437
Total Path Length              : 516
Total Path Cost                : 516.00
Total Execution Time           : 224.83 ms
------------------------------
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 23:21:30
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Breadth-First Search

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 34)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 5
Success Rate                   : 100.0%
Number of States Expanded      : 1922
Number of States Generated     : 5621
Total Path Length              : 158
Total Path Cost                : 158.00
Total Execution Time           : 26.38 ms
------------------------------
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 23:21:42
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Breadth-First Search

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 38)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 5
Success Rate                   : 100.0%
Number of States Expanded      : 4662
Number of States Generated     : 11546
Total Path Length              : 239
Total Path Cost                : 239.00
Total Execution Time           : 70.23 ms
------------------------------
```

Lampiran Bidirectional Search

Run ke-	Makanan Terkumpul	Success Rate (%)	States Expanded	States Generated	Path Length	Path Cost	Exec Time (ms)
1	5	100	4896	19150	536	536	59.93
2	5	100	10660	41102	1120	1120	141.29
3	5	100	2230	8716	361	361	20.15
4	5	100	5659	22288	639	639	74.05
5	5	100	5517	21712	632	632	60.49
Average (Avg)	**5**	**100**	**5792.4**	**22593.6**	**657.6**	**657.6**	**71.182**

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 23:08:59
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Bidirectional Search

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 59)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 5
Success Rate                   : 100.0%
Number of States Expanded      : 4896
Number of States Generated     : 19150
Total Path Length              : 536
Total Path Cost                : 536.00
Total Execution Time           : 59.93 ms
----------------------------
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 23:09:28
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Bidirectional Search

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 97)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 5
Success Rate                   : 100.0%
Number of States Expanded      : 10660
Number of States Generated     : 41102
Total Path Length              : 1120
Total Path Cost                : 1120.00
Total Execution Time           : 141.29 ms
----------------------------
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 23:09:46
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Bidirectional Search

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 53)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 5
Success Rate                   : 100.0%
Number of States Expanded      : 2230
Number of States Generated     : 8716
Total Path Length              : 361
Total Path Cost                : 361.00
Total Execution Time           : 20.15 ms
----------------------------
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 23:10:08
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Bidirectional Search

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 75)

--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 5
Success Rate                   : 100.0%
Number of States Expanded      : 5659
Number of States Generated     : 22288
Total Path Length              : 639
Total Path Cost                : 639.00
Total Execution Time           : 74.05 ms
----------------------------
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 23:10:57
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : Bidirectional Search

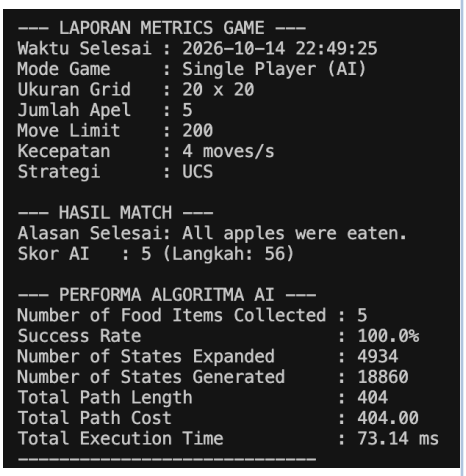
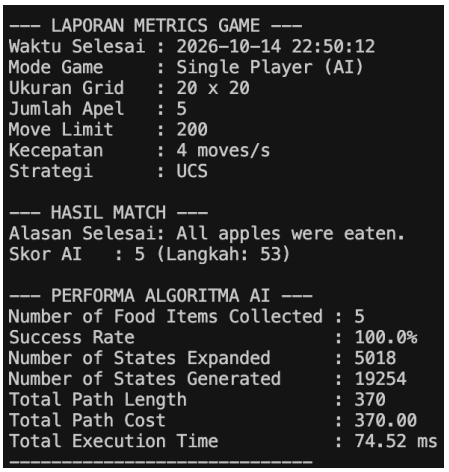
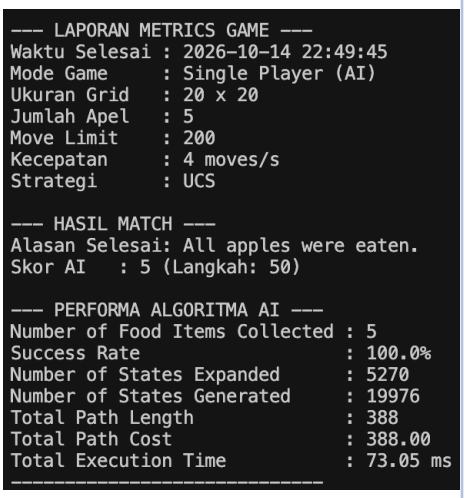
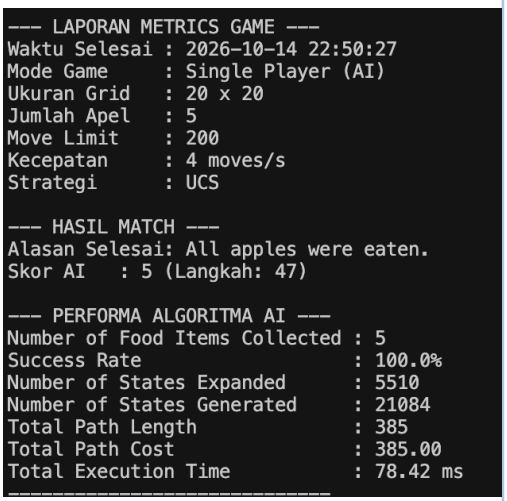
--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 73)

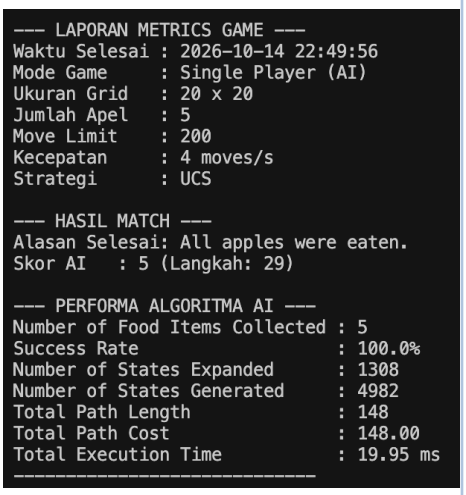
--- PERFORMA ALGORITMA AI ---
Number of Food Items Collected : 5
Success Rate                   : 100.0%
Number of States Expanded      : 5517
Number of States Generated     : 21712
Total Path Length              : 632
Total Path Cost                : 632.00
Total Execution Time           : 60.49 ms
----------------------------
```

Lampiran UCS

Run ke-	Makanan Terkumpul	Success Rate (%)	States Expanded	States Generated	Path Length	Path Cost	Exec Time (ms)
1	5	100.0%	4934	18860	404	404.00	73.14
2	5	100.0%	5270	19976	388	388.00	73.05
3	5	100.0%	1308	4982	148	148.00	19.95
4	5	100.0%	5018	19254	370	370.00	74.52
5	5	100.0%	5510	21084	385	385.00	78.42
Average (Avg)	**5**	**100.0%**	**4408**	**16831.2**	**339**	**339.00**	**63.82**

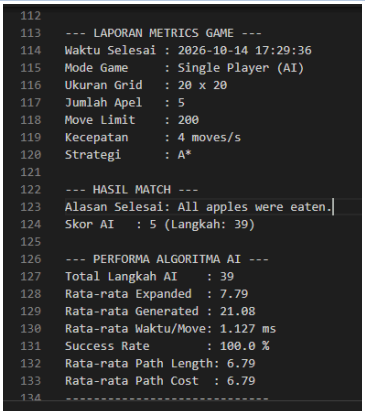
SCREENSHOT METRICS

Percobaan ke-	Ringkasan Match	Percobaan ke-	Ringkasan Match
1		4	
2		5	

Percobaan ke-	Ringkasan Match	Percobaan ke-	Ringkasan Match
3			

Lampiran A*

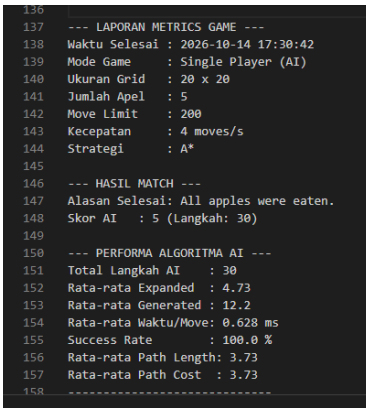
Run ke-	Makanan Terkumpul	Success Rate (%)	States Expanded	States Generated	Path Length	Path Cost	Exec Time (ms)
1	5	100.0%	7.79	21.08	6.79	6.79	1.127
2	5	100.0%	4.73	12.2	3.73	3.73	0.628
3	5	100.0%	8.6	22.96	7.6	7.6	1.148
4	5	100.0%	5.63	13.95	4.63	4.63	0.805
5	5	100.0%	7.08	17.64	6.08	6.08	0.821
Average (Avg)	**5**	**100.0%**	**6.766**	**17.566**	**5.766**	**5.766**	**0.9058**



```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 17:29:36
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : A*

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 39)

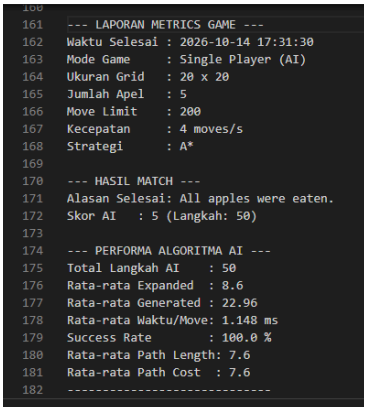
--- PERFORMA ALGORITMA AI ---
Total Langkah AI    : 39
Rata-rata Expanded  : 7.79
Rata-rata Generated : 21.08
Rata-rata Waktu/Move: 1.127 ms
Success Rate        : 100.0 %
Rata-rata Path Length: 6.79
Rata-rata Path Cost  : 6.79
```



```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 17:30:42
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : A*

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 30)

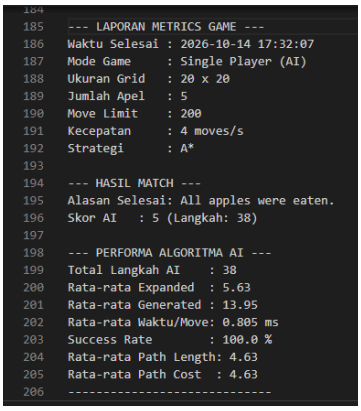
--- PERFORMA ALGORITMA AI ---
Total Langkah AI    : 30
Rata-rata Expanded  : 4.73
Rata-rata Generated : 12.2
Rata-rata Waktu/Move: 0.628 ms
Success Rate        : 100.0 %
Rata-rata Path Length: 3.73
Rata-rata Path Cost  : 3.73
```



```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 17:31:30
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : A*

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 50)

--- PERFORMA ALGORITMA AI ---
Total Langkah AI    : 50
Rata-rata Expanded  : 8.6
Rata-rata Generated : 22.96
Rata-rata Waktu/Move: 1.148 ms
Success Rate        : 100.0 %
Rata-rata Path Length: 7.6
Rata-rata Path Cost  : 7.6
```



```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 17:32:07
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : A*

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 38)

--- PERFORMA ALGORITMA AI ---
Total Langkah AI    : 38
Rata-rata Expanded  : 5.63
Rata-rata Generated : 13.95
Rata-rata Waktu/Move: 0.805 ms
Success Rate        : 100.0 %
Rata-rata Path Length: 4.63
Rata-rata Path Cost  : 4.63
```

```
--- LAPORAN METRICS GAME ---
Waktu Selesai : 2026-10-14 17:32:47
Mode Game     : Single Player (AI)
Ukuran Grid   : 20 x 20
Jumlah Apel   : 5
Move Limit    : 200
Kecepatan     : 4 moves/s
Strategi      : A*

--- HASIL MATCH ---
Alasan Selesai: All apples were eaten.
Skor AI   : 5 (Langkah: 53)

--- PERFORMA ALGORITMA AI ---
Total Langkah AI    : 53
Rata-rata Expanded  : 7.08
Rata-rata Generated : 17.64
Rata-rata Waktu/Move: 0.821 ms
Success Rate        : 100.0 %
Rata-rata Path Length: 6.08
Rata-rata Path Cost  : 6.08
------------------------------
```

Lampiran MINIMAX (as Player 1)

Depth = 1

Algoritma	Depth	Win Rate (%)	Avg Score	States Generated	States Evaluated	Pruned Branches	Exec Time / Move (ms)
Minimax	d=1	Win	3	58	58	0	2,89
Minimax	d=1	Lose	1	72	72	0	3,58
Minimax	d=1	Win	3	71	71	0	3,37
Minimax	d=1	Win	4	84	84	0	4,09
Minimax	d=1	Win	3	51	51	0	4,58
Average		**80%**	**2,8**	**67,2**	**67,2**	**0**	**3,70**

Depth = 2

Algoritma	Depth	Win Rate (%)	Avg Score	States Generated	States Evaluated	Pruned Branches	Exec Time / Move (ms)
Minimax	d=2	Win	3	362	267	0	10.07
Minimax	d=2	Win	4	355	263	0	09.13
Minimax	d=2	Win	3	374	279	0	10.40
Minimax	d=2	Win	3	390	292	0	10.23
Minimax	d=2	Win	3	424	315	0	10.35
Average		**100%**	**3,2**	**381**	**283,2**	**0**	**10.11**

Depth = 3

Algoritma	Depth	Win Rate (%)	Avg Score	States Generated	States Evaluated	Pruned Branches	Exec Time / Move (ms)
Minimax	d=3	Lose	1	799	547	0	16,51
Minimax	d=3	Win	3	1371	923	0	25,16
Minimax	d=3	Win	3	701	481	0	13,44
Minimax	d=3	Lose	2	773	531	0	14,25
Minimax	d=3	Win	3	1491	1021	0	28,66
Average		**60%**	**2,4**	**1027**	**700,6**	**0**	**14.29**

Depth = 4

Algoritma	Depth	Win Rate (%)	Avg Score	States Generated	States Evaluated	Pruned Branches	Exec Time / Move (ms)
Minimax	d=4	Win	4	2052	1381	0	34,43
Minimax	d=4	Win	3	2297	1530	0	39,02
Minimax	d=4	Win	3	2345	1560	0	38,77
Minimax	d=4	Win	3	4056	2641	0	66,82
Minimax	d=4	Lose	2	1939	1293	0	31,43
Average		**80%**	**3**	**2537,8**	**1681**	**0**	**42,09**

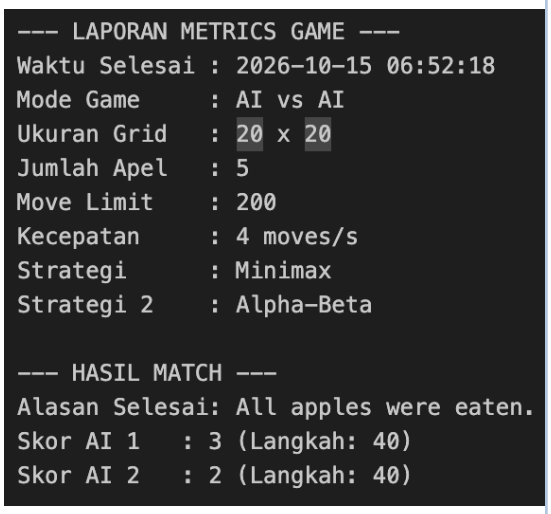
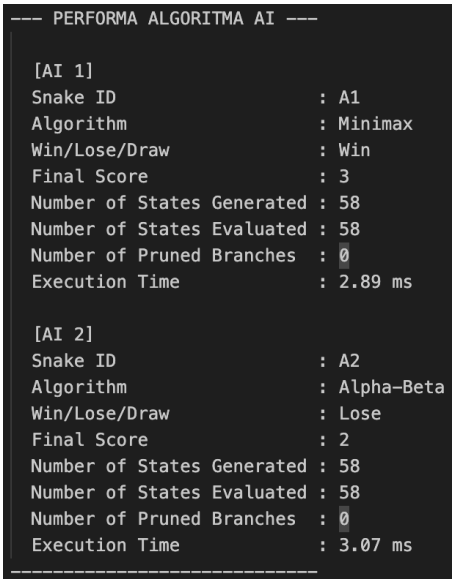
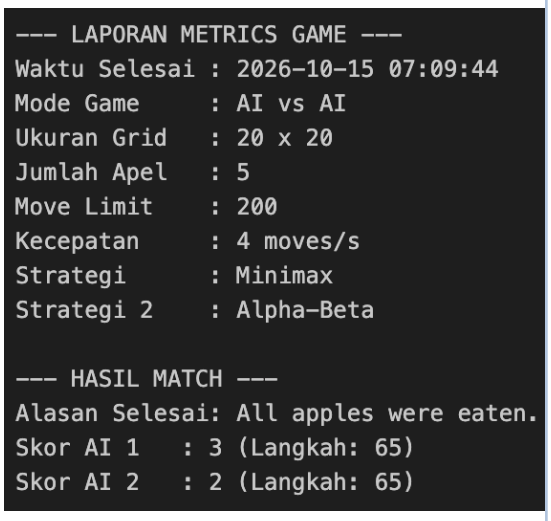
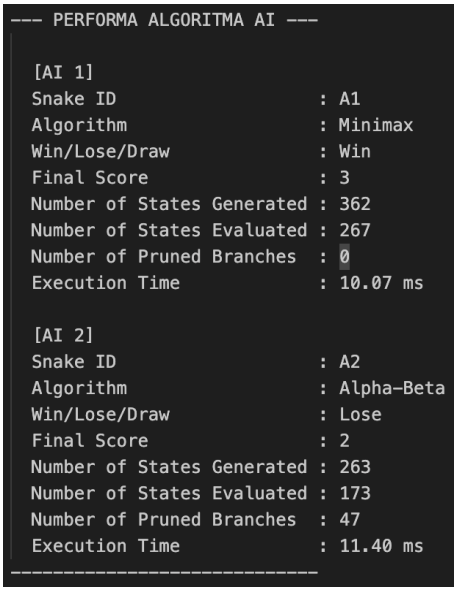
Depth = 5

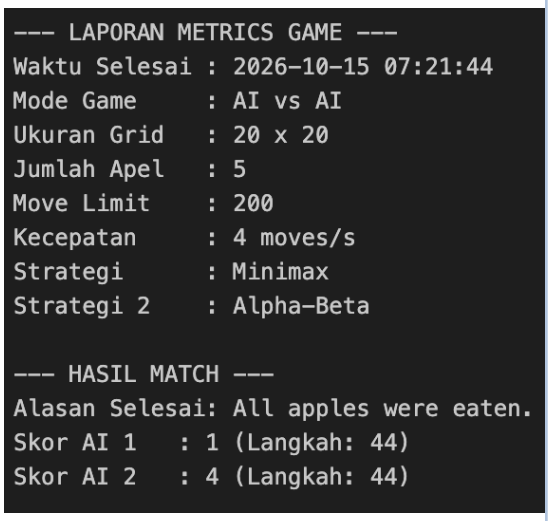
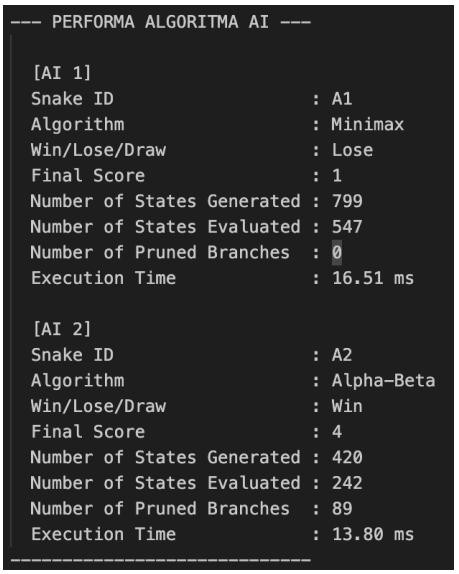
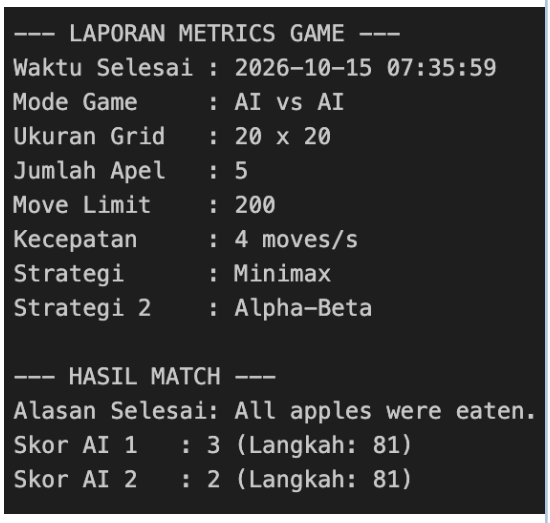
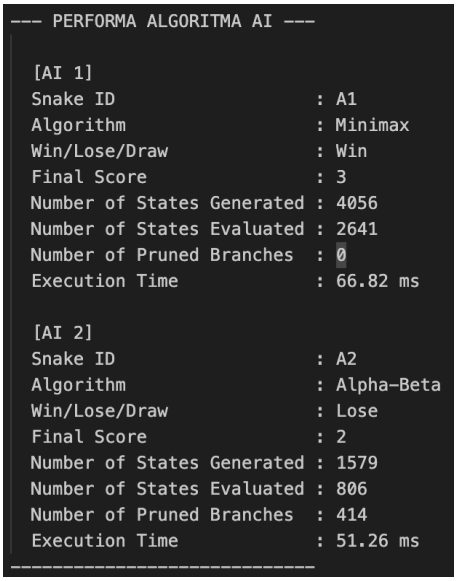
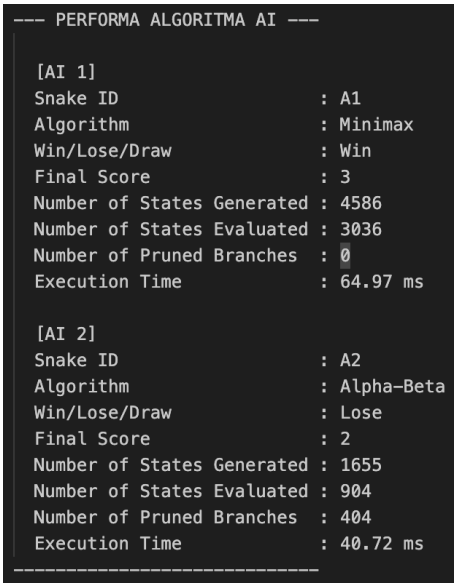
Algoritma	Depth	Win Rate (%)	Avg Score	States Generated	States Evaluated	Pruned Branches	Exec Time / Move (ms)
Minimax	d=5	Win	3	6757	4427	0	96,56
Minimax	d=5	Win	3	10577	6876	0	149,40
Minimax	d=5	Win	3	4586	3036	0	64,97
Minimax	d=5	Win	3	8088	5235	0	111,28
Minimax	d=5	Win	3	6819	4522	0	99,04
Average		**100%**	**3**	**7365,4**	**4819,2**	**0**	**104,25**

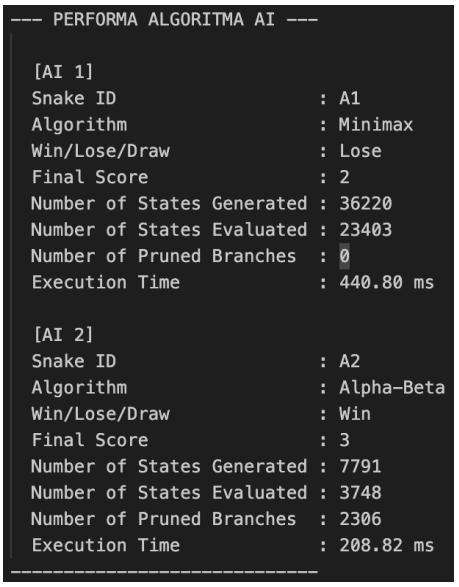
Depth = 6

Algoritma	Depth	Win Rate (%)	Avg Score	States Generated	States Evaluated	Pruned Branches	Exec Time / Move (ms)
Minimax	d=6	Lose	2	24394	15961	0	283,42
Minimax	d=6	Lose	2	36220	23403	0	440,80
Minimax	d=6	Win	3	21127	13806	0	257,40
Minimax	d=6	Win	3	22872	14921	0	272,50
Minimax	d=6	Win	3	31616	20727	0	383,87
Average		**60%**	**2,6**	**27245,8**	**17763,6**	**0**	**327,60**

SCREENSHOT METRICS (Perwakilan)

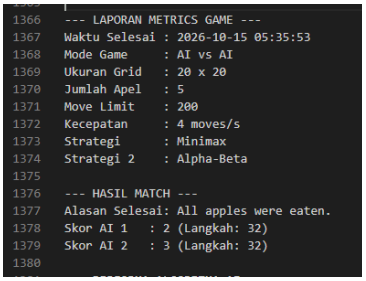
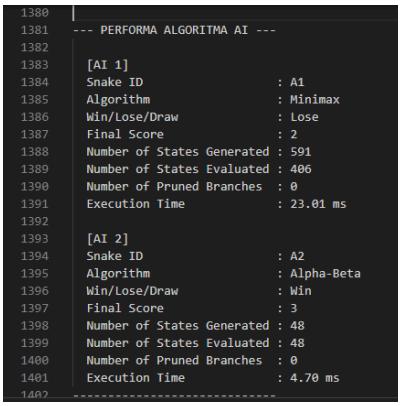
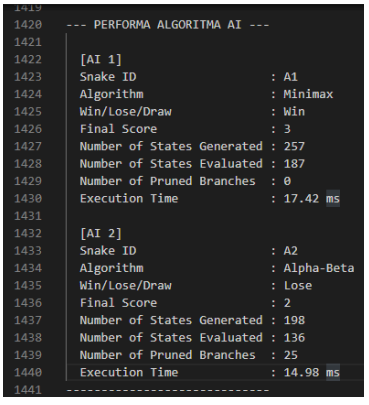
Depth	Ringkasan Match	Performa AI (Minimax as Player 1)
1		
2		

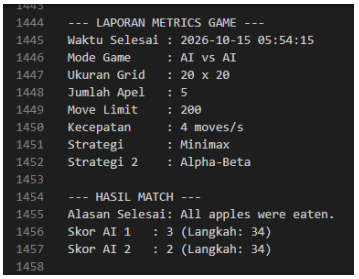
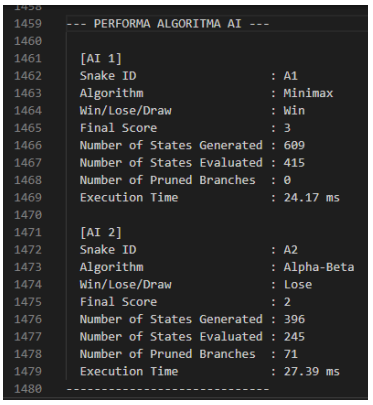
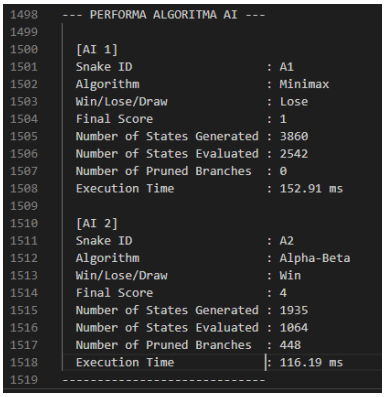
Depth	Ringkasan Match	Performa AI (Minimax as Player 1)
3		
4		
5		

Depth	Ringkasan Match	Performa AI (Minimax as Player 1)
6		

Lampiran AlphaBeta Pruning

Algoritma	Depth	Win Rate (%)	Avg Score	States Generated	States Evaluated	Pruned Branches	Exec Time / Move (ms)
Alpha-beta	d=1	3/5	3	48	48	0	4.70ms
Alpha-beta	d=2	2/5	2	198	136	25	14.98ms
Alpha-beta	d=3	2/5	2	396	245	71	27.39ms
Alpha-beta	d=4	4/5	4	1935	1064	448	116.19ms
Alpha-beta	d=5	2/5	2	1570	870	386	103.39ms
Average (Avg)		**2.6/5 (52%)**	**2.6**	**829.4**	**472.6**	**186**	**53.33**

Depth	Picture 1	Picture 2
1		
2		

Depth	Picture 1	Picture 2
3		
4		
5		